

can

由于一家做汽车电子的公司面试我的时候，问我有没有学过CAN，我说没有，我之前也听过CAN，但是就是没学，所以这段时间就学习了一下

根据江科大的CAN入门教程，加上自己的理解整理而成，CAN协议真的很完美，感谢大佬的讲解！

CAN简介

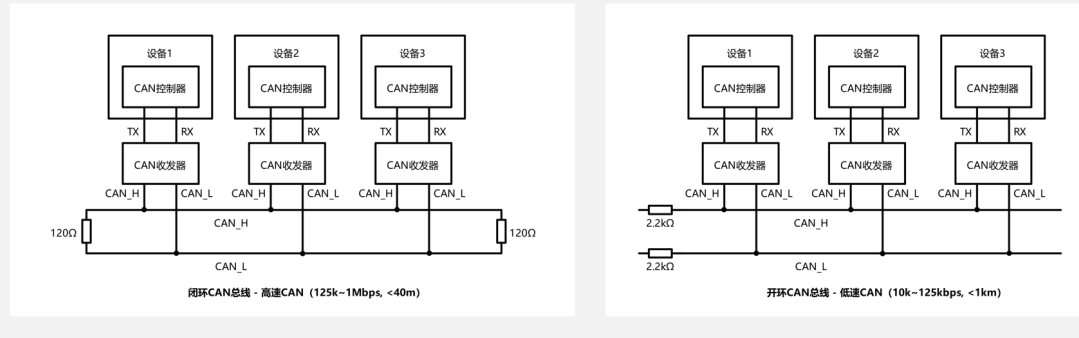
- CAN总线（Controller Area Network Bus）控制器局域网总线
- CAN总线是由BOSCH公司开发的一种简洁易用、传输速度快、易扩展、可靠性高的串行通信总线，广泛应用于汽车、嵌入式、工业控制等领域
- CAN总线特征：
 - 两根通信线（CAN_H、CAN_L），线路少
 - 差分信号通信，抗干扰能力强
 - 高速CAN（ISO11898）：125k~1Mbps, <40m
 - 低速CAN（ISO11519）：10k~125kbps, <1km
 - 异步，无需时钟线，通信速率由设备各自约定
 - 半双工，可挂载多设备，多设备同时发送数据时通过仲裁判断先后顺序
 - 11位/29位报文ID，用于区分消息功能，同时决定优先级
 - 可配置1~8字节的有效载荷
 - 可实现广播式和请求式两种传输方式
 - 应答、CRC校验、位填充、位同步、错误处理等特性

主流通信协议对比

名称	引脚	双工	时钟	电平	设备	应用场景
UART	TX、RX	全双工	异步	单端	点对点	两个设备互相通信
I2C	SCL、SDA	半双工	同步	单端	多设备	一个主控外挂多个模块
SPI	SCK、MOSI、MISO、SS	全双工	同步	单端	多设备	一个主控外挂多个模块（高速）
CAN	CAN_H、CAN_L	半双工	异步	差分	多设备	多个主控互相通信

CAN应用场景，多个stm32互相通信，比如，10个哈哈

- 每个设备通过**CAN收发器**挂载在CAN总线网络上
- CAN控制器引出的TX和RX与CAN收发器相连，CAN收发器引出的CAN_H和CAN_L分别与总线的CAN_H和CAN_L相连
- 高速CAN使用**闭环**网络，CAN_H和CAN_L两端添加**120Ω**的终端电阻
- 低速CAN使用**开环**网络，CAN_H和CAN_L其中一端添加**2.2kΩ**的终端电阻



这里为什么要加电阻？

闭环的设计120欧

1.没有设备工作的时候，将两根差分线 电压收紧，使电压一致，就是设备没有操作总线的时候，将两根线像弹簧一样收紧，拉倒同一水平，为1的状态，阻值越小，拉力越强，设备操纵总线 时候就拉开，就是为0的状态

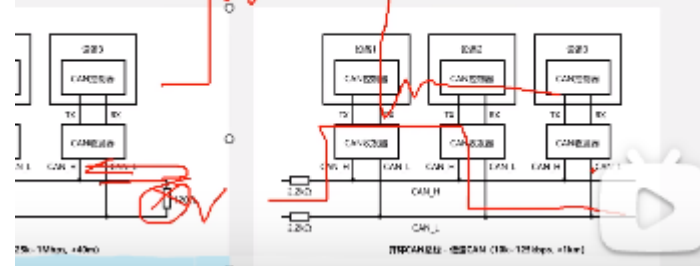
如图：



2.防止回波反射，尤其是高频信号，远距离传输的场景，不加的话，信号回波会在线路中断反射，进而干

扰。CAN_H和CAN_L两端添加120Ω的终端电阻
 网络。CAN_H和CAN_L其中一端添加2.2kΩ的终端电阻

扰原始信号，就是有干扰，如图：

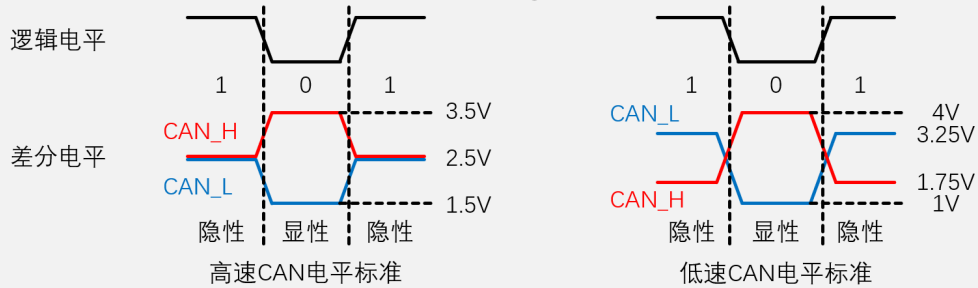


第二章 帧格式

可以想象成鼓掌，双手合起来，代表1，双手分开，代表0

1 CAN电平标准

- CAN总线采用差分信号，即两线电压差 ($V_{CAN_H} - V_{CAN_L}$) 传输数据位
- 高速CAN规定：
 - 电压差为0V时表示逻辑1（隐性电平）
 - 电压差为2V时表示逻辑0（显性电平）
- 低速CAN规定：
 - 电压差为-1.5V时表示逻辑1（隐性电平）
 - 电压差为3V时表示逻辑0（显性电平）

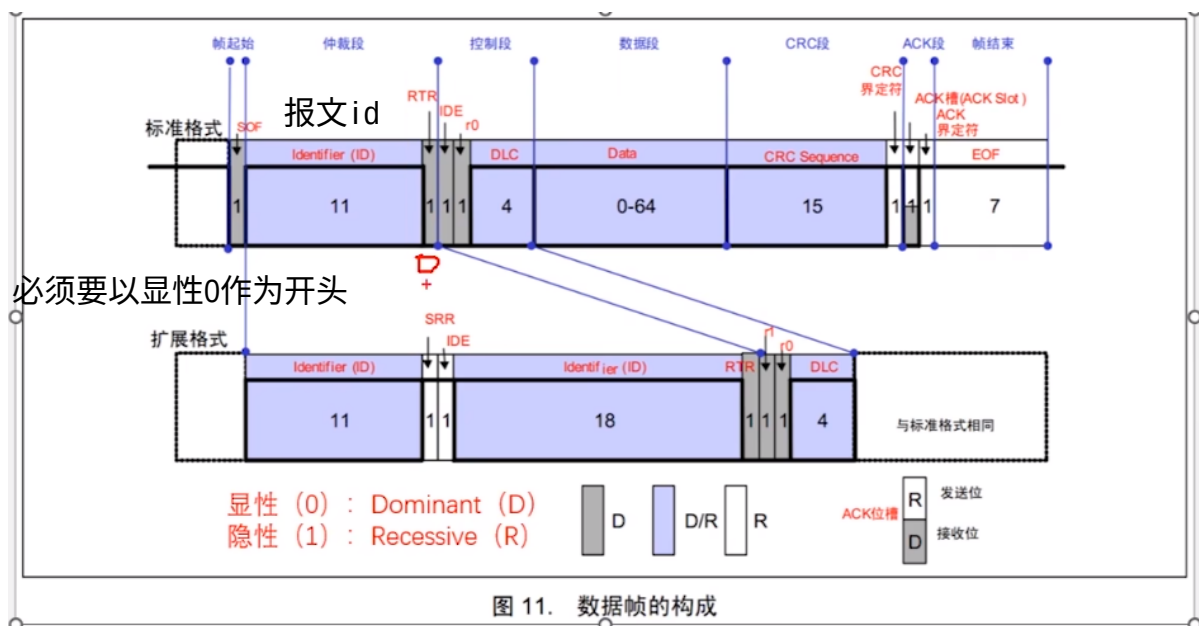


0强于1，是自然收紧必须要外力才能张开

2.发送帧格式

帧类型	用途
数据帧	发送设备主动发送数据（广播式）
遥控帧	接收设备主动请求数据（请求式）
错误帧	某个设备检测出错误时向其他设备通知错误
过载帧 I	接收设备通知其尚未做好接收准备
帧间隔	用于将数据帧及遥控帧与前面的帧分离开

数据帧



注意，里面的数字，11,1,1,1,4,64....表示的是数量长度，比如那个4就是1000，，

帧起始，拉低0，因为总线空闲是1隐形的，要区分---

0.id(11位)，报文id 表示后面数据功能或是区分优先级

注意：ID的优先级，越小越高，是采用的十进制比较或十六进制比较

id大的优先发送，id小的等待下一次总线空闲发送

1.RTR:远程请求标志，

用于区分是数据帧还是遥控帧的，

数据帧必须为显性0，

遥控帧必须为隐性1

数据帧和遥控帧相同情况下，数据帧的优先级大于遥控帧

ID+RTR是仲裁段

跟12c一样，然后就是7位地址+1位读/写，id就是地址，RTR就是数据帧（写入），遥控帧（读取）；

IIC这里是7位指定地址+1位指定读或写

CAN这里改成了对报文ID进行分配：这样同一个报文可以被多个设备接收，

2.IDE (r1): 扩展格式，主要是区分谁是标准格式，谁是扩展格式,方便后续升级

标准格式是显示0，扩展格式是隐性1

3.r0就是保留位 为了分辨是标准格式还是扩展格式

必须是0；

0的优先级更高

后续要用到

4.DLC

dLC 就是指data数据长度，因为can总线的一帧传的数据长度是1~8个字节，

比如想发送一个字节就是0001，如果想发送8个就想1000，

DLC指定多少字节数据段就发送多少字节

5.Data就是数据的长度 0~64位，就是最大8个字节，

然后就是，必须是每次都是8个位的发送，因为，一个字节是8个比特呀

6.CRC校验

校验码，这个是计算得到的

7.CRC界定符，必须是隐形1

因为要给ack应答，有应答了就是0，得区分一下，不然一直是0，有没有应答都不知道

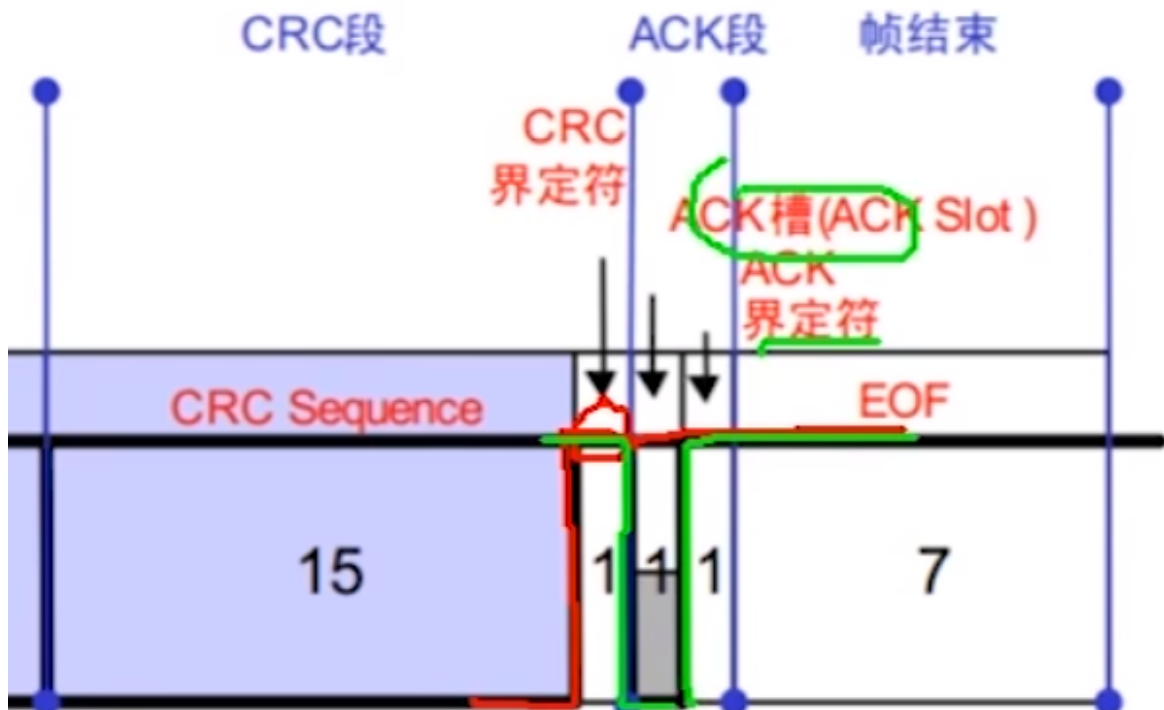
8.ACK应答槽

发送释放 接收拉开 接收释放

发送方发送隐形1（第一个隐形符），然后发送过去，

然后交接权利，让接收方应答

如果是接收方收到了就会把ack应答给拉低为0，然后再释放（第二个界定符），然后发送方在检验ack位置，如果是1，他不开心，就说明失败了，要么可以重发，要么可以不发



注意事项

(1) 允许多个接收方接收发送方的消息，共同拉开总线，多个拉开显示0，就是拉低显示0，绿色那里没有问题的

切记，在ACK槽之前都是发送方发信；在ACK槽时是接收方操作总线应答置零的。
可以多个设备同时操作拉开总线。

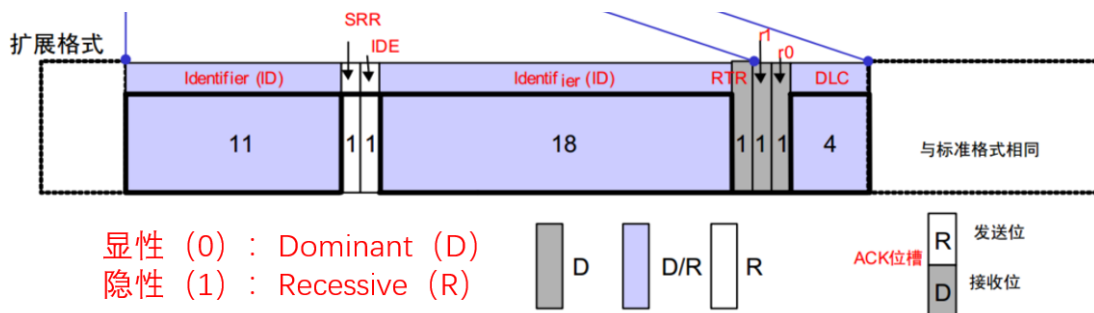
接收, 给0, 再拉手返回

(2) 并不是发送方单方面发送消息的，不是发送完在应答，而是边发送边应答，**发送方发送一位，接收方就收到一位了**，所以整个数据帧还没有结束就，接收方就收完了

9.eof发送7个1，结束

扩展模式

出现原因是因为11位ID不够用了



这里只是比标准模式多了srr和18位id

标准模式：仲裁是先比较ID再比较RTR，RTR必须在ID后面。

srr,就是占位的跟标准模式的RTR类似，但是这里要区别标准模式，**标准模式优先级>扩展模式**，

这里要用隐性为1，

扩展格式后RTR挪到最后去了，原来的RTR就空出来了，但是必须要给1，为了保证标准>扩展优先级。

然后IDE也要为1，表示扩展位置

然后这里因为是扩展模式了，RTR后面的IDE就恢复原状r1了。

2 数据帧各部分用途简介



- SOF (Start of Frame)：帧起始，表示后面一段波形为传输的数据位
- ID (Identify)：标识符，区分功能，同时决定优先级
- RTR (Remote Transmission Request)：远程请求位，区分数据帧和遥控帧
- IDE (Identifier Extension)：扩展标志位，区分标准格式和扩展格式
- SRR (Substitute Remote Request)：替代RTR，协议升级时留下的无意义位
- r0/r1 (Reserve)：保留位，为后续协议升级留下空间
- DLC (Data Length Code)：数据长度，指示数据段有几个字节
- Data：数据段的1~8个字节有效数据
- CRC (Cyclic Redundancy Check)：循环冗余校验，校验数据是否正确
- ACK (Acknowledgement)：应答位，判断数据有没有被接收方接收
- CRC/ACK界定符：为应答位前后发送方和接收方释放总线留下时间
- EOF (End of Frame)：帧结束，表示数据位已经传输完毕

遥控帧

请求式的，有来有回

就是，先发送个遥控帧，然后如果其他设备有这个id的数据的话，其他设备回一个数据帧，然后接收方就收到了数据

请求这个遥控id的数据，请求和数据帧同时发生时数据帧优先级更高

一次完整的数据请求，需要遥控帧和数据帧的配合，请求和发送同时发生时，数据帧优先级更大

- 遥控帧无数据段，RTR为隐性电平1，其他部分与数据帧相同

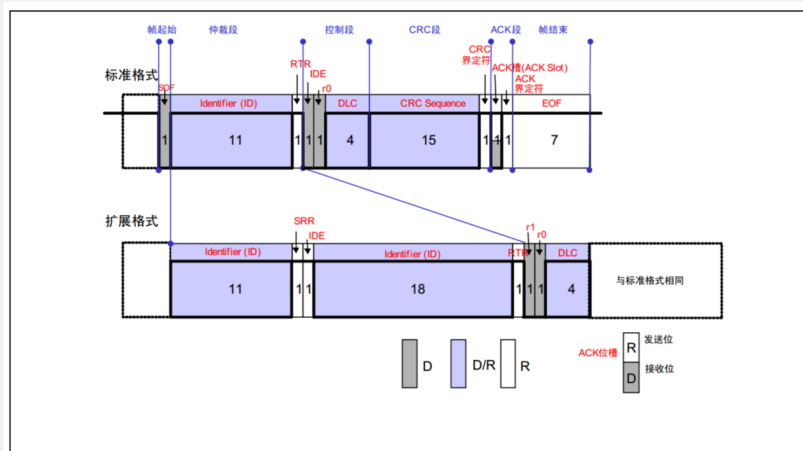


图 12. 遥控帧的构成

错误帧

2 错误帧

- 总线上所有设备都会监督总线的数据，一旦发现“位错误”或“填充错误”或“CRC错误”或“格式错误”或“应答错误”，这些设备便会发出错误帧来破坏数据，同时终止当前的发送设备

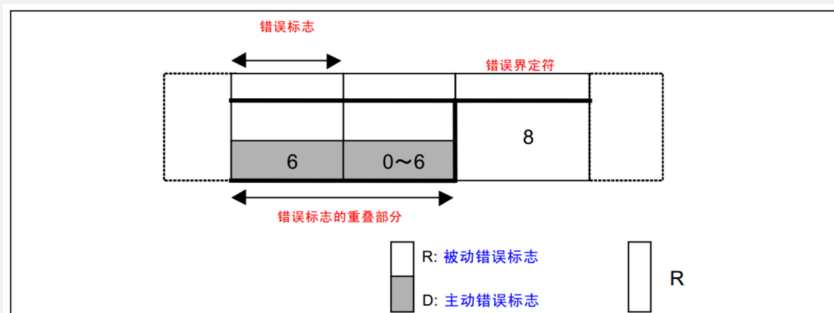


图 13. 错误帧

错误帧可以叠加在数据帧上，也可以破坏数据帧的数据

设备默认是主动错误状态，检测到错误时，会发6个显性位置0，就是拉开总线，

主动错误频繁了，就会觉得不可靠，就会进入被动状态，发6个隐性位1，就是不碰总线，不会破坏总线的数据，但是会破坏自己的数据

然后再发8个隐性1,错误界定符

那为什么还要画0~6个错误标志延长位勒？

因为：就是设备1的错误信息最后一位开始，才被设备2检测，设备2才开始发6位干扰数据，就0-6了呗

过载帧 由接收方产生，因为发送方发太快所致。

2 错误帧



- 总线上所有设备都会监督总线的数据，一旦发现“位错误”或“填充错误”或“CRC 错误”或“格式错误”或“应答错误”，这些设备便会发出错误帧来破坏数据，同时终止当前的发送设备

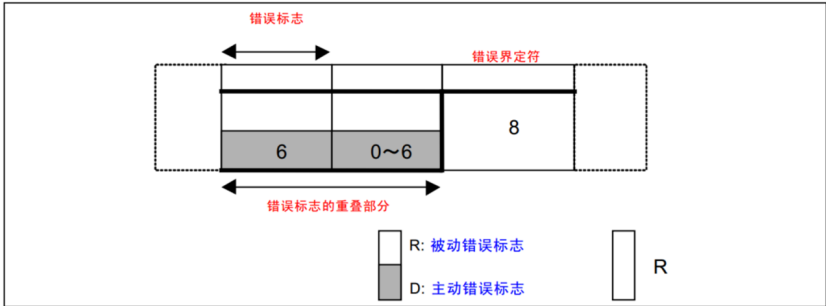


图 13. 错误帧

跟错误帧类似

帧间隔

2 帧间隔



- 将数据帧和遥控帧与前面的帧分离开

上面是主动错误状态+3位，
下面是被动错误状态+3位+8位延迟

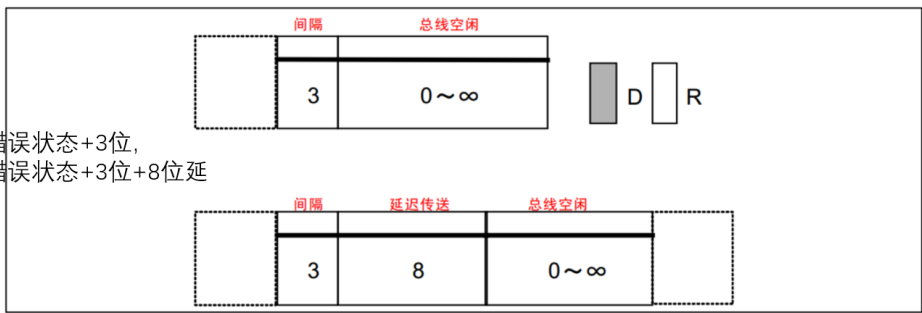


图 15. 帧间隔

延迟传输降低优先级，避免干扰总线

位填充

会在波形中显示出来，crc计算的时候要剔除填充位

位填充机制只存在于开始到CRC界定符之间

因为是29位，所以，范围是0x0000 0000~0x1FFF FFFF,最前面的是0（就是第一位）

比如扩展数据帧，报文ID为0x0789ABCD

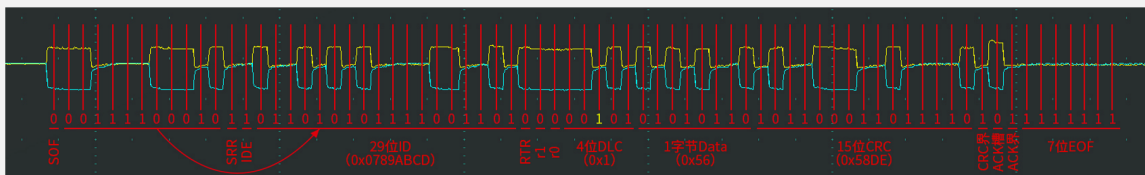
二进制是：0111 1000 1001 1010 1011 1100 1101，但是只有28位，要补个0 占位

就是：00111 1000 1001 1010 1011 1100 1101

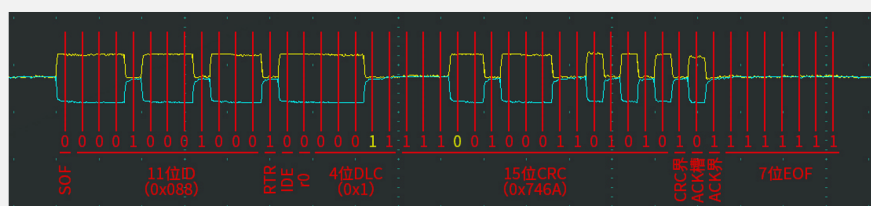
2 波形实例



- 扩展数据帧，报文ID为0x0789ABCD，数据长度1字节，数据内容为0x56



- 标准遥控帧，报文ID为0x088，数据长度1字节，无数据内容



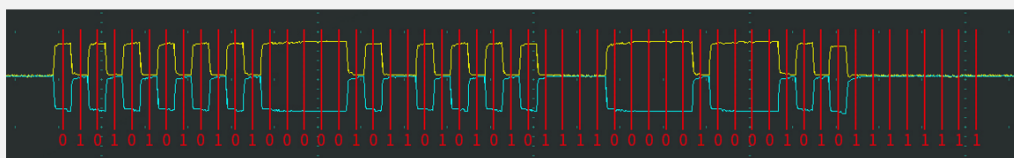
怎么接收了，就需要相应的电路设计了，上面 的只是方便连接

第三章 位同步

3 接收方数据采样



- CAN总线没有时钟线，总线上的所有设备通过约定波特率的方式确定每一个数据位的时长
- 发送方以约定的位时长每隔固定时间输出一个数据位
- 接收方以约定的位时长每隔固定时间采样总线的电平，输入一个数据位
- 理想状态下，接收方能依次采样到发送方发出的每个数据位，**且采样点位于数据位中心附近**



SS应当是电平变换的位置，当然在SS处电平也可以不发生改变，如果电平变化不在SS处，则需用硬同步和再同步校准

- 每个设备都有一个位时序计时周期，当某个设备（发送方）率先发送报文，其他所有设备（接收方）收到SOF的下降沿时，接收方会将自己的位时序计时周期拨到SS段的位置，与发送方的位时序计时周期保持同步
- 硬同步只在帧的第一个下降沿（SOF下降沿）有效
- 经过硬同步后，若发送方和接收方的时钟没有误差，则后续所有数据位的采样点必然都会对齐数据位中心附近

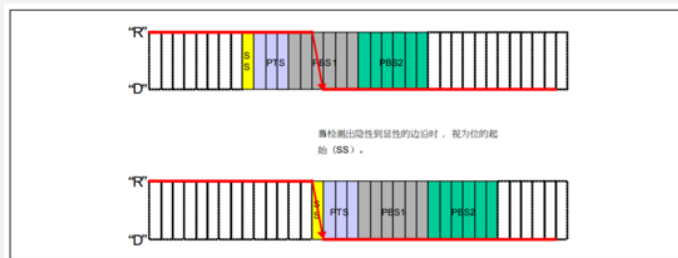


图 33. 硬件同步



你看：红色的线，发送方从高电平到低电平，这就是sof，然后就是，接收方，就是那些格子，ss段没有和红色的下降沿重合（上图），所以就是自动移位重合了一下（下图）

sof:第一个下数据跳变的边沿

硬同步只在帧的第一个下降沿（逻辑电平的下降沿，差分电平的张开沿）

发送方出现第一个下降边沿，然后就是ss段，然后就是接收方就知道了，也调到ss段，就将所有的秒表同步了

采样是在PBS1和PBS2之间交接处采样

然后就是接收方和发送方都有一个'秒表'，当转动到ss就说明同步了，然后就可以正常传送消息

但是，还是有点误差的，你知道吗，就是一会正常，后面就不正常了，所以就是下面的再同步

再同步

就是补偿误差，

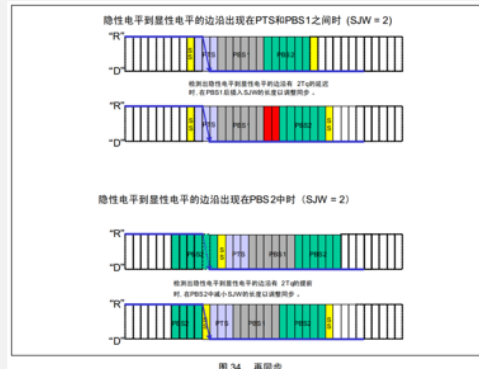
误差补偿只能由接收方来做。

注意这里sjw是最大补偿值，而不是每次都补偿规定的数，防止波形中的噪声对位时序造成过大影响

比如SJW=2,这里快了1个位置，还是只补偿一个位置，而不是2个，是要误差格子和SJW共同商量，

如果误差>SJW,都只补偿SJW最大的值

- 若发送方或接收方的时钟有误差，随着误差积累，数据位边沿逐渐偏离SS段，则此时接收方根据再同步补偿宽度值（SJW）通过加长PBS1段，或缩短PBS2段，以调整同步
- 再同步可以发生在第一个下降沿之后的每个数据位跳变边沿



➤ $SJW = 1 \sim 4Tq$

分析：

上图是接收方快于发送方，下图是接收方慢于发送方 **接收方快了：加补偿 Tq**

1.上面两个图，就是比如先进行了一次硬同步，然后正常了一会，。然后就是又跑偏了，你看那个红色的下降沿都没有跟黄色的ss段重合，就是有误差了，要么是接收方的秒表跑快了，要么是发送方跑慢了。总之现在sahib接收方快于发送方

但是，发送方是固定的速度呀，不可能调整熟读的呀，只能接收方来调整，进行误差补偿，上图就红色那，加了两个延时，数据就往后延时了，采样就会在中间，没有就不在中间了

注意：数据采样是在PBS1和PBS2之间 采样的，

接收方慢了：PBS2减少 Tq

2.你看这里就是接收方慢于发送方，发送方提前了，所以这里减少误差补偿就是2个 Tq ，然后就是你看左边绿色的PBS2是不是就是**少了两个方格**

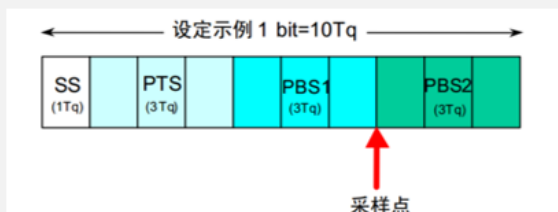
注意，硬同步只能在第一个下降沿一次，二再同步可以在第一个下降沿的每一个数据段

波特率计算

每秒传输多少个bit

在二进制调制下，波特率=比特率的值

- 波特率 = $1 / \text{一个数据位的时长} = 1 / (T_{SS} + T_{PTS} + T_{PBS1} + T_{PBS2})$
- 例如：
 $SS = 1T_q$, $PTS = 3T_q$, $PBS1 = 3T_q$, $PBS2 = 3T_q$
 $T_q = 0.5\mu s$
 波特率 = $1 / (0.5\mu s + 1.5\mu s + 1.5\mu s + 1.5\mu s) = 200\text{kbps}$



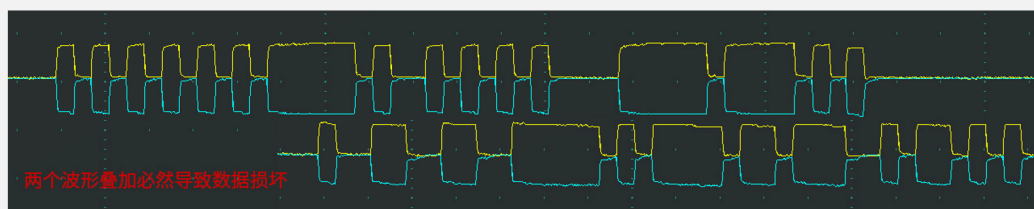
这里除了SS是1Tq,其他的都我们自己设置的

第四章 仲裁

线与：只要有一个设备拉开总线，就是0，所有设备释放了总线，才是1

4 多设备同时发送遇到的问题

- CAN总线只有一对差分信号线，同一时间只能有一个设备操作总线发送数据，若多个设备同时有发送需求，该如何分配总线资源？
- 解决问题的思路：制定资源分配规则，依次满足多个设备的发送需求，确保同一时间只有一个设备操作总线



问题：

上面的那个东西发送方先发了，然后又有一个发送方想发送东西，

这时候要么是不让第二个走，就是方案一

要么是把第二个移到前面去，和第一个发送方是在同一个起始位置然后这里就是方案二，

4 资源分配规则1 - 先占先得

- 若当前已经有设备正在操作总线发送数据帧/遥控帧，则其他任何设备不能同时发送数据帧/遥控帧（可以发送错误帧/过载帧破坏当前数据）
- 任何设备检测到连续11个隐性电平，即认为总线空闲，只有在总线空闲时，设备才能发送数据帧/遥控帧
- 一旦有设备正在发送数据帧/遥控帧，总线就会变为活跃状态，必然不会出现连续11个隐性电平，其他设备自然也不会破坏当前发送
- 若总线活跃状态其他设备有发送需求，则需要等待总线变为空闲，才能执行发送需求

4 资源分配规则2 - 非破坏性仲裁

- 若多个设备的发送需求同时到来或因等待而同时到来，则CAN总线协议会根据ID号（仲裁段）进行非破坏性仲裁，ID号小的（优先级高）取到总线控制权，ID号大的（优先级低）仲裁失利后将转入接收状态，等待下一次总线空闲时再尝试发送
- 实现非破坏性仲裁需要两个要求：
 - 线与特性：总线上任何一个设备发送显性电平0时，总线就会呈现显性电平0状态，只有当所有设备都发送隐性电平1时，总线才呈现隐性电平1状态，即： $0 \& X \& X = 0$ ， $1 \& 1 \& 1 = 1$
 - 回读机制：每个设备发出一个数据位后，都会读回总线当前的电平状态，以确认自己发出的电平是否被真实地发送出去了，根据线与特性，发出0读回必然是0，发出1读回不一定是1（因为设备1发送了1，然后设备2发送了0，然后那不就是线与最后就是0了嘛，所以数据错误）

4 非破坏性仲裁过程

- 数据位从前到后依次比较，出现差异且数据位为1的设备仲裁失利

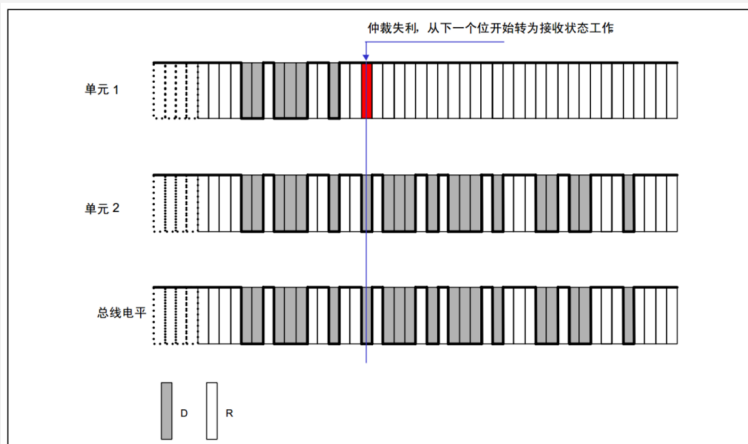


图 28. 仲裁过程

出现1说明id越大，即优先级越低。

执行流程：发送一位，回读一位 发一位回读一位，发的是0必然是0，发的是1因为线与可能不一定是1，所以一旦回读的和发送的不一致则自动退出仲裁抢夺。

你看，这个单元1和单元2前面（红色之前那段）都是一样的，他们发送的消息，然后总线返回的数据跟他们发送的一样的，那么就会继续，（比如发送的是1，然后总线的也是1，然后设备读取到的数据回读机制），然后到了红色那，设备1发送1，设备2发送0，然后根据的是设备id优先级仲裁，设备号小的就优先级大，然后（ $0 > 1$ ），还有线与的功能，此时总线是0，然后最后回读，设备1发送的是1，但读到总线是0，那么失败了，设备2发送0，读取到0，成功

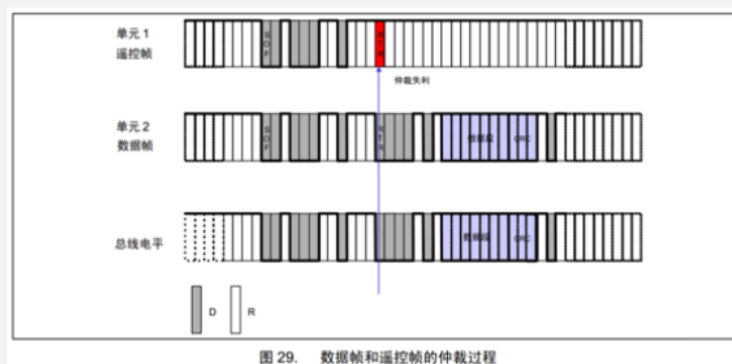
在仲裁里会设置位填充，那么位填充会不会影响优先级的啊，不会影响，

位填充会参与仲裁，但是不会影响，不会改变原有的id号的优先级

4 数据帧和遥控帧的优先级



- 数据帧和遥控帧ID号一样时，数据帧的优先级**高于**遥控帧



两个相同的设备的 数据帧不能进行仲裁

两个相同的设备的 遥控帧不能进行仲裁

两个相同的数据帧和遥控帧可以进行仲裁

- 标准格式11位ID号和扩展格式29位ID号的高11位一样时，标准格式的优先级高于扩展格式（**SRR必须始终为1**，以保证此要求）**标准帧>扩展帧**

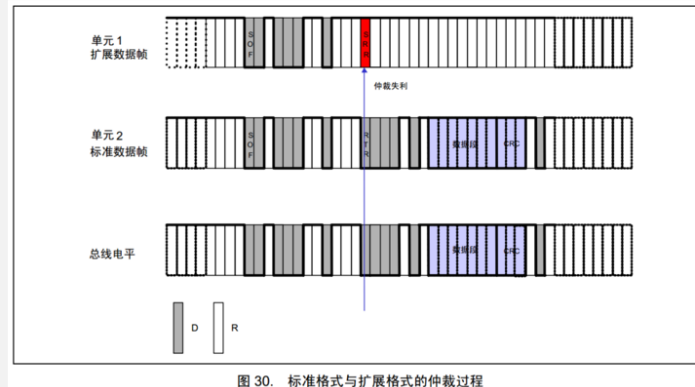


图 30. 标准格式与扩展格式的仲裁过程

你看，如果这里是标准遥控帧和扩展数据帧进行仲裁，又该如何操作？

遥控帧RTR是1，扩展数据帧的SRR也是1，然后标准帧就结束了仲裁，但是还有个IDE，数据帧是0，扩展数据帧因为扩展了就是1，所以线与之了，扩展帧出现发1收0的情况，然后就自动退出了，所以标准遥控帧获胜

第五章 错误处理

5 错误类型

- 错误共有5种：位错误、填充错误、CRC错误、格式错误、应答错误

表 9. 错误的种类

错误的种类	错误的内容	错误的检测帧（段）	检测单元
位错误	比较输出电平和总线电平（不含填充位），当两电平不一样时所检测到的错误。	- 数据帧（SOF-EOF） - 遥控帧（SOF-EOF） - 错误帧 - 过载帧	发送单元 接收单元
填充错误	在需要位填充的段内，连续检测到6位相同的电平时所检测到的错误。	- 数据帧（SOF-CRC顺序） - 遥控帧（SOF-CRC顺序）	发送单元 接收单元
CRC错误	从接收到的数据计算出的CRC结果与接收到的CRC顺序不同时所检测到的错误。	- 数据帧（CRC顺序） - 遥控帧（CRC顺序）	接收单元
格式错误	检测出与固定格式的位段相反的格式时所检测到的错误。	- 数据帧（CRC界定符、ACK界定符、EOF） - 遥控帧（CRC界定符、ACK界定符、EOF） - 错误界定符 - 过载界定符	接收单元
ACK错误	发送单元在ACK槽(ACK Slot)中检测出隐性电平时所检测到的错误（ACK没被传过来时所检测到的错误）。	- 数据帧（ACK槽） - 遥控帧（ACK槽）	发送单元

位错误：就是回读机制，设备发送的数据，然后读取总线的的数据，然后对比，一样就没有问题，但是，不一样就是有位错误。

但是排除仲裁段，（只是仲裁失败，不是发送失败），还有就是应答机制（发1收0），他只是应答，不是位错误

格式错误：就是CRC界定符，ACK界定符，EOF...这些都是1，但是如果被检测到时0，那就是格式错误

如果发送方或者接收方检测到这些错误，就会发送错误通知，就会停止数据，终止。然后总线就变空闲，等到下一次数据传输

当然，现在每个设备都可以破坏数据传输，但是设备要是发疯，那么不严谨，一直出错，所以得设置防止一直出错，然后就是有了**设备的错误状态**

5 错误状态

- 主动错误状态的设备正常参与通信并在检测到错误时发出**主动**错误帧
- 被动错误状态的设备正常参与通信**但检测到错误时**只能发出**被动**错误帧
- 总线**关闭**状态的设备**不能参与通信**
- 每个设备内部管理一个TEC和REC，根据TEC和REC的值确定自己的状态

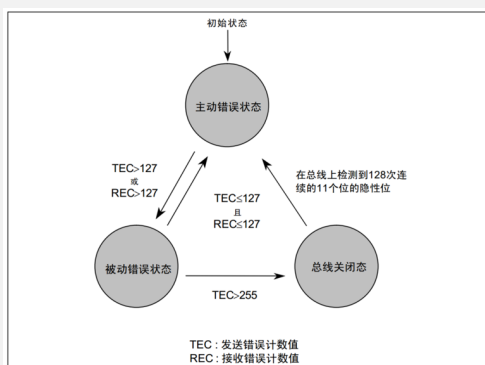


图4. 单元的错误状态

一开始是**主动错误状态**，会拉点为0，有破坏数据的能力，但是太频繁了就会变成**被动的错误状态**，然后就是错误的转态，是1，就是释放总线，没有破坏数据的能力，但是要是也频繁了，就会被关闭了。哈哈

关闭了，但是想要启动，就得总线上检测到128次11个隐形就可以启动了。

信誉分

TEC是发送的错误状态值，如果检测到错误的话，就+1，但是如果发送成功一次的话就-1，

REC是接收的错误状态值，如果检测到错误的话，就+1，但是如果发送成功一次的话就-1，

所有他们反应的是当前设备检测的状态错误的相对频率

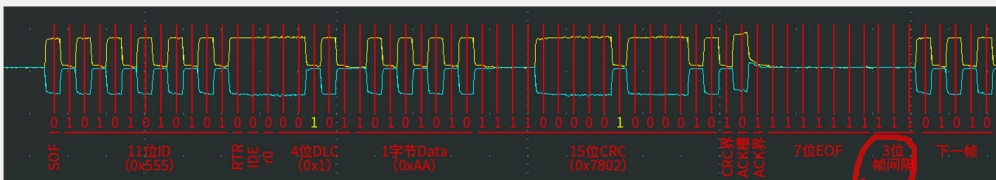
TEC REC小于127后会回到主动错误状态；但如果设备在TEC大于255时会转移成关闭状态。检测到128次11个隐性位会复活，清零TEC REC。

表 2. 错误计数值的变动条件

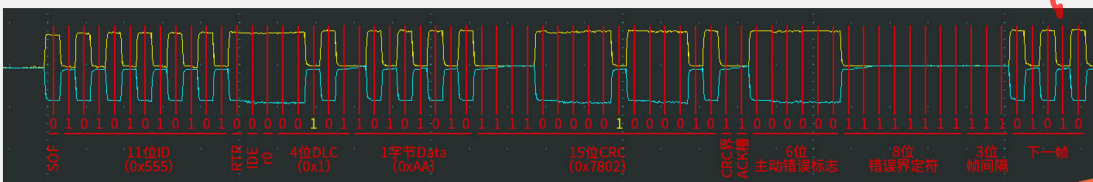
	接受和发送错误计数值的变动条件	发送错误计数值 (TEC)	接收错误计数值 (REC)
1	接收单元检测出错误时。 例外：接收单元在发送错误标志或过载标志中检测出“位错误”时，接收错误计数值不增加。	—	+1
2	接收单元在发送完错误标志后检测到的第一个位为显性电平时。	—	+8
3	发送单元在输出错误标志时。	+8	—
4	发送单元在发送主动错误标志或过载标志时，检测出位错误。	+8	—
5	接收单元在发送主动错误标志或过载标志时，检测出位错误。	—	+8
6	各单元从主动错误标志、过载标志的最开始检测出连续 14 个位的显性位时。 之后，每检测出连续的 8 个位的显性位时。	发送时 +8	接收时 +8
7	检测出在被动错误标志后追加的连续 8 个位的显性位时。	发送时 +8	接收时 +8
8	发送单元正常发送数据结束时（返回 ACK 且到帧结束也未检测出错误时）。	-1 TEC=0 时±0	—
9	接收单元正常接收数据结束时（到 CRC 未检测出错误且正常返回 ACK 时）。	—	1≤REC≤127 时-1 REC=0 时±0 REC>127 时 设 REC=127
10	处于总线关闭态的单元，检测到 128 次连续 11 个位的隐性位。	TEC=0	REC=0

5 波形示例

- 设备处于主动错误状态，发送标准数据帧，正常传输



- 设备处于主动错误状态，发送标准数据帧，检测到ACK错误



6个显性0 | 8个隐性1

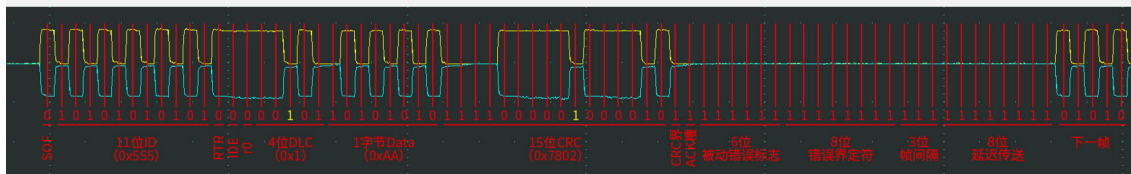
正常结束了，eof后面还有3个帧间隔，是给过渡帧用的，如果过载了，就会在第一个帧间隔拉开总线，产生过渡帧。

第一个最上面正常标志：ack1位+7位EOF+3位帧间隔=11位，所以每个设备检测到11位隐性电平就是空闲状态。

第1个图下面：

8位错误界定符+3位帧间隔=11位隐性

- 设备处于**被动错误状态**，发送标准数据帧，**检测到ACK错误**



第2个图，这里因为主动状态从**16次**后就变成了**被动状态**，然后就，所以那个过渡帧，就会额外产生8位延迟

因为 $127/8=16$;

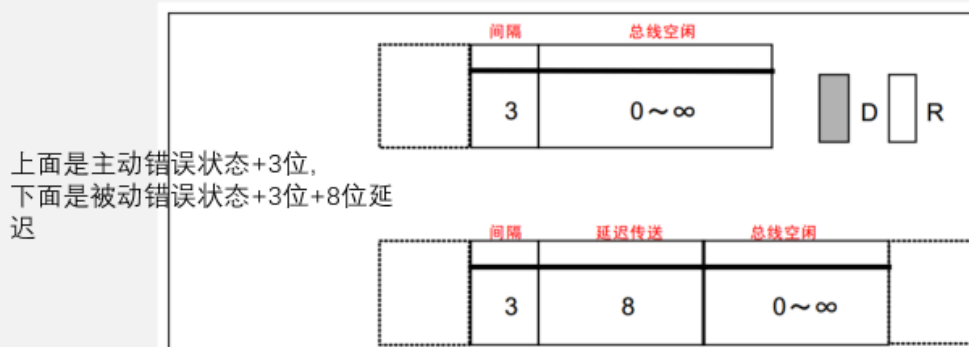
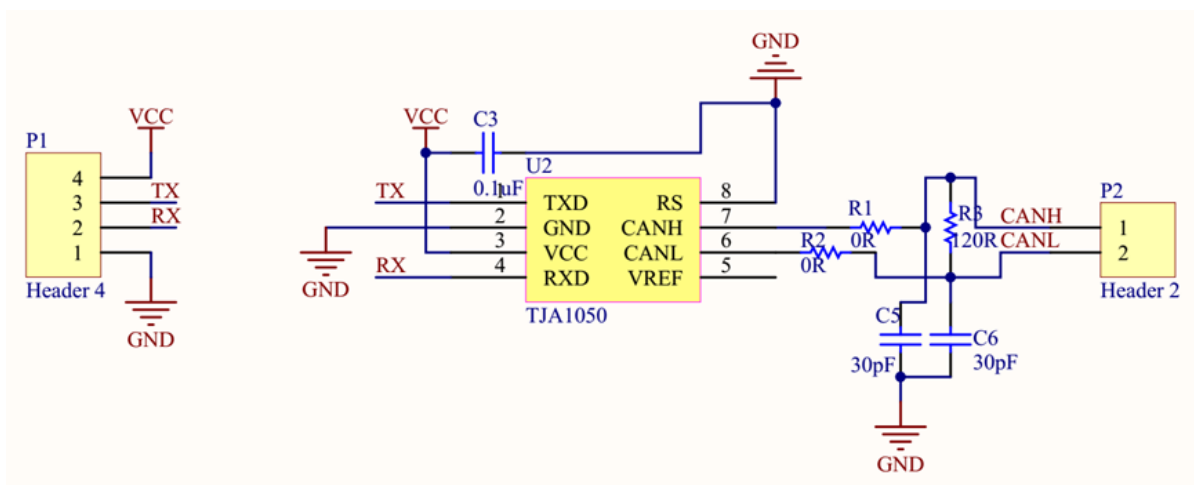


图 15. 帧间隔

如果主动错误状态和被动错误状态，同时处于空闲时，那么主动错误状态会直接控制总线，因为被动状态还有8个延时

stm32 can 外设 上

接收器，这里直接rx-rx,tx-tx,注意要区分跟串口不一样，这里不能交叉连接；是因为接收器的设计导致的。

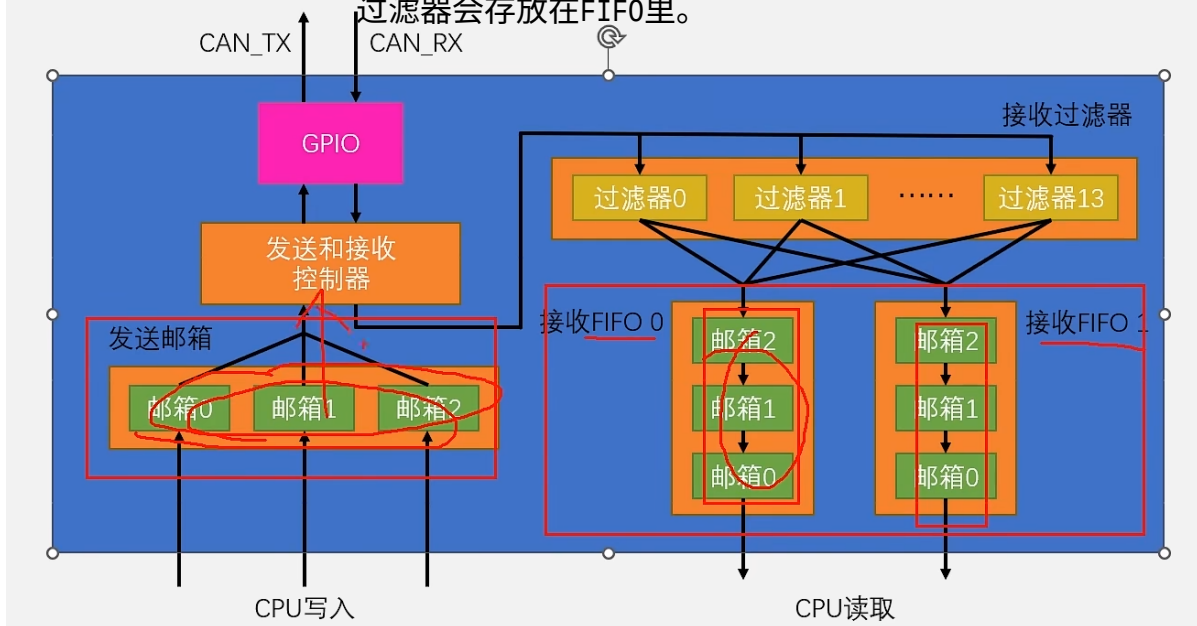


TX：复用推挽输出；RX：上拉输入

2 CAN基本结构

需要告诉控制器发出什么报文，写入到邮箱里，给出发送命令；以及接受什么报文，接受的报文经过过滤器会存放在FIFO里。

江协科
jiangxiekeji



发送，可以配置按id优先级发送，也可以按先后来后到发送

接收就只能是队列，先来先到

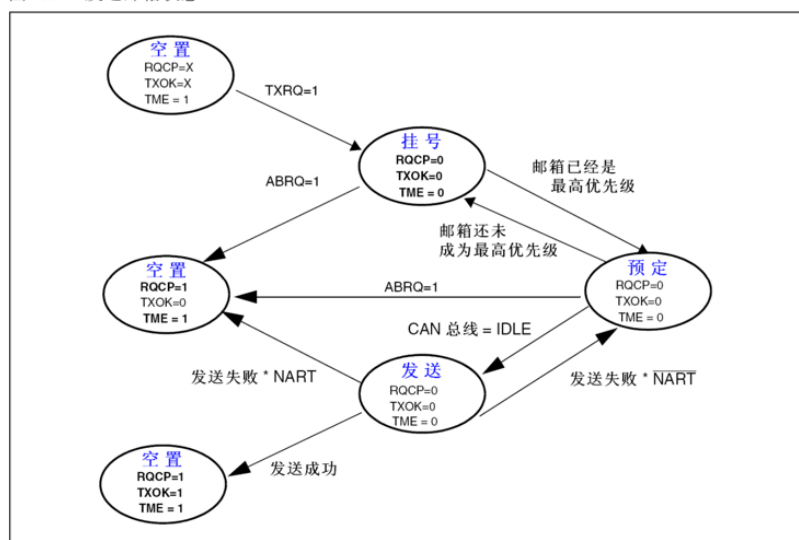
为什么这里要设置两个队伍呀？

因为就是，就好比是饭堂打饭，就是，学生的窗口和老师的窗口，有重要的和不重要的嘛，当然这里是相等的优先级

2 发送过程

- 基本流程：选择一个空置邮箱→写入报文→请求发送

图200 发送邮箱状态



RQCP: 请求完成

TXOK: 发送成功

TME: 发送邮箱空, =1表示空闲状态

x:表示任意值

TXRQ: 发送请求控制位, =1表示产生这个请求

IDLE空闲

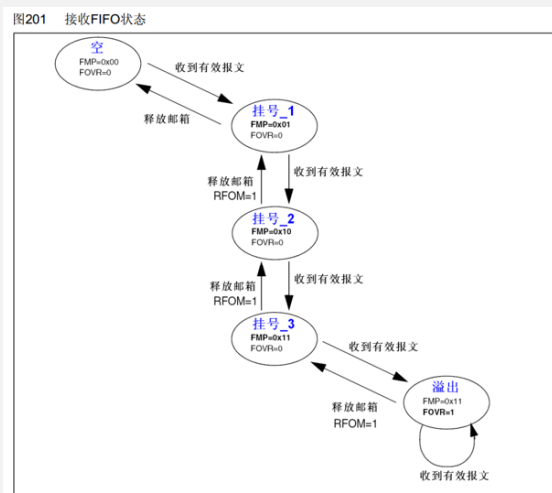
ABRQ: 终止该发送

NART:禁止自动重传, =0表示使用自动重传, 就回到预定转态然后等待总线空闲, 然后再次发送, =1表示禁止自动重传, 发送失败后, 直接进入空置状态

2 接收过程



- 基本流程: 接收到一个报文→匹配过滤器后进入FIFO 0或FIFO 1→CPU读取



溢出和挂号3状态是一个状态。退出溢出状态后会直接进到挂号2

FMP:报文数目 0=00, 第一个报文就是0先, 第二个就是0x10b,第三个就是0x11b,溢出了也是0x11b

FIOVR:FIFO溢出为1, 没有溢出就是0

溢出了再次收到消息也都是溢出

RFOM=1, 读完一个报文就要释放邮箱

如果是溢出状态, 那么就是直接跳转到报文2, 而不是从溢出跳转到报文3, 再到报文2, 溢出时状态三就是跟状态一样的。

- **NART:**
 - 置1, 关闭自动重传, CAN报文只被发送1次, 不管发送的结果如何(成功、出错或仲裁丢失);
 - 置0, 自动重传, CAN硬件在发送报文失败时会一直自动重传直到发送成功
- **TXFP:**
 - 置1, 优先级由发送请求的顺序来决定, 先请求的先发送;
 - 置0, 优先级由报文标识符来决定, 标识符值小的先发送(标识符值相等时, 邮箱号小的报文先发送)
- **RFLM:**
 - 置1, 接收FIFO锁定, FIFO溢出时, 新收到的报文会被丢弃;
 - 置0, 禁用FIFO锁定, FIFO溢出时, FIFO中最后收到的报文被新报文覆盖

can 外设下

2 标识符过滤器

- 每个过滤器的核心由两个32位寄存器组成: R1[31:0]和R2[31:0]
- **FSCx:** 位宽设置
置0, 16位; 置1, 32位
- **FBMx:** 模式设置
置0, 屏蔽模式; 置1, 列表模式
-----上面三个寄存器设置了什么可以通过哪些报文, 什么不能通过哪些报文-----
- **FFAx:** 关联设置, 就是报文进fifo 0 还是fifo1
置0, FIFO 0; 置1, FIFO 1
- **FACTx:** 激活设置, 就是启用这个过滤器
置0, 禁用; 置1, 启用

图202 过滤器组位宽设置—寄存器组织

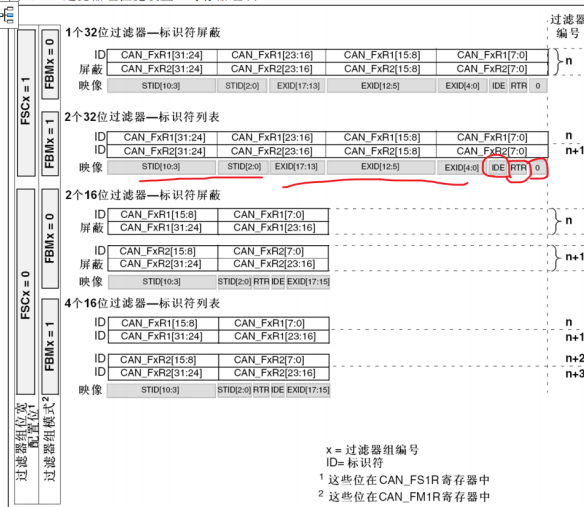
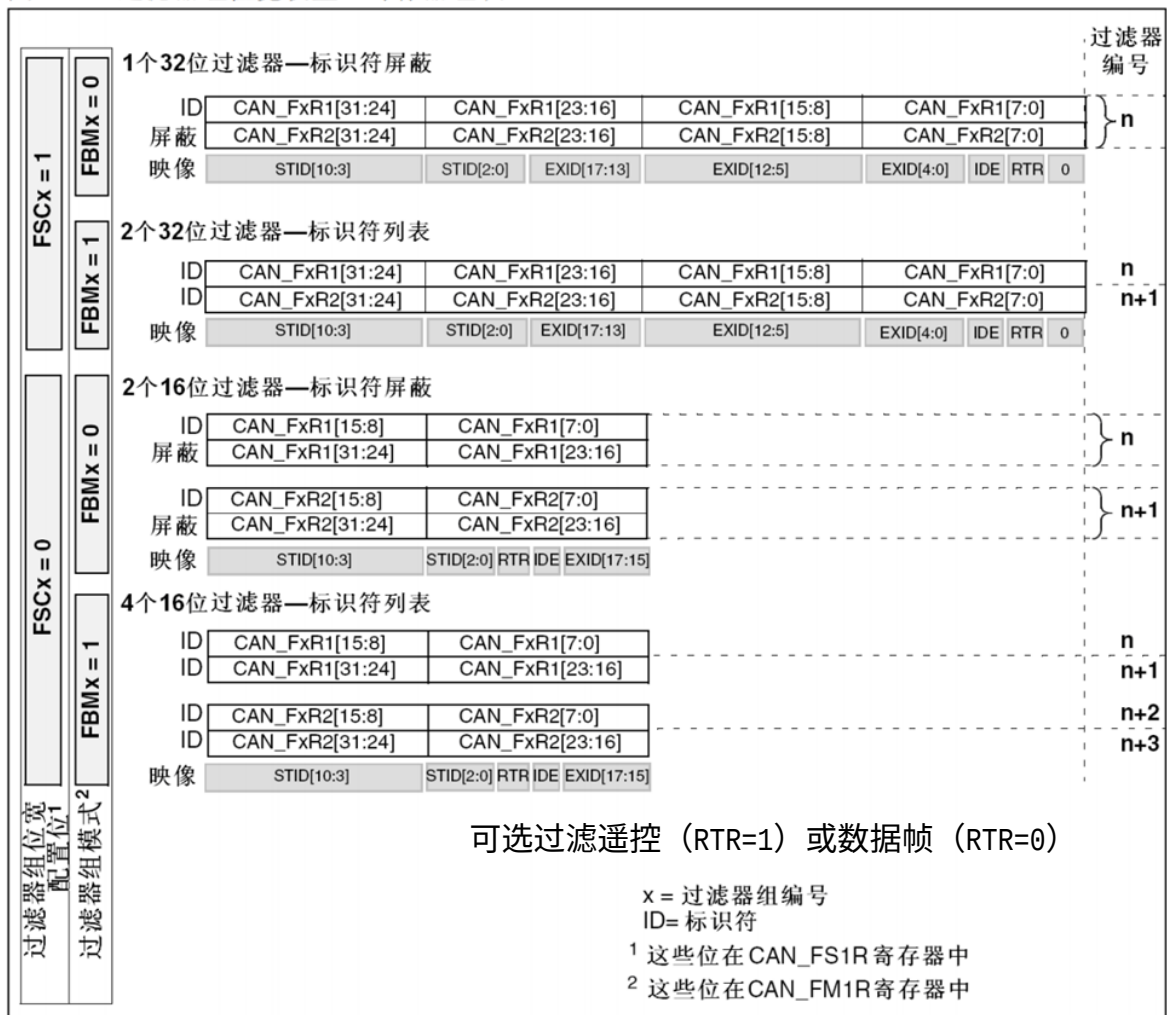


图202 过滤器组位宽设置—寄存器组织



32位：映像那：如果是标准id，那就是填11位（如0x123<<21），然后后面的扩展id就填0不管，然后后面的IDE为0，然后再RTR遥控帧的数据

如果是扩展的话，那就是要占用前面的11位，然后还有就是后面的18位，然后就是IDE要为1，如果填了0，那就只能当做标准位来实现

16位过滤器，被拆开了，可以有4个过滤名单，但是只能是标准格式的，

列表：直接写进寄存器

屏蔽：就是按组写，就比如，一个传感器，温度有100个数据探头，，湿度有100个数据探头，

然后分组定义，温度是0x100,0x101,0x101.....

湿度是0x200,0x201,0x202....

然后就是，按组屏蔽，如果是高位是1，就是温度，高位是2就是湿度，id的低位可以改变

这个屏蔽位，就像网络一样，R1为网络号 R2为掩码 两者相与得id

如列表给了个标准id，001 0000 0000

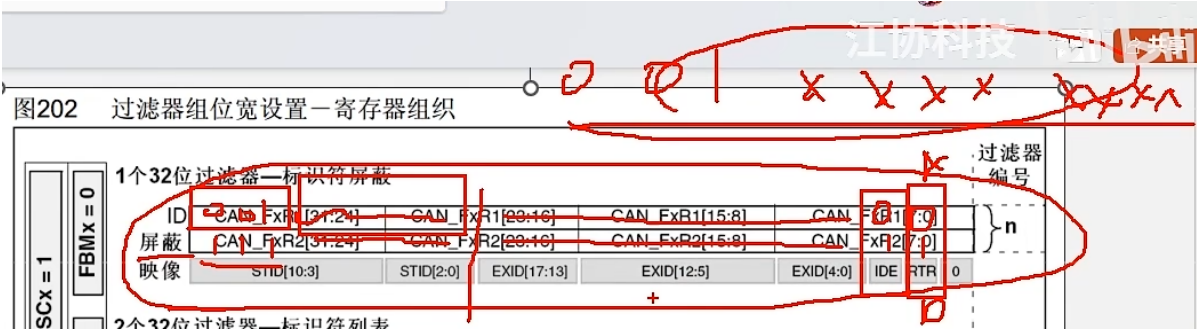
屏蔽位给0，就是和列表R1对应位为0和1 都可以

如果给1，就是必须和列表R1对应位一样才可以，

那么就是屏蔽位高3位都给1*才可以过**，后面的8位随便就可以过

然后后面的IDE也要给1，表示和上面的R1寄存器IDE必须匹配，如果屏蔽位给了0，表示标准格式和扩展格式都无所谓，，然后就是会导致某些扩展id误入进来，不符合需求

然后RTR,也是，与R!一样就是给1，都可以就是给0，比如R!的RTR给了1，那么R2的RTR想一致的话给1，遥控帧或数据帧都要就给0



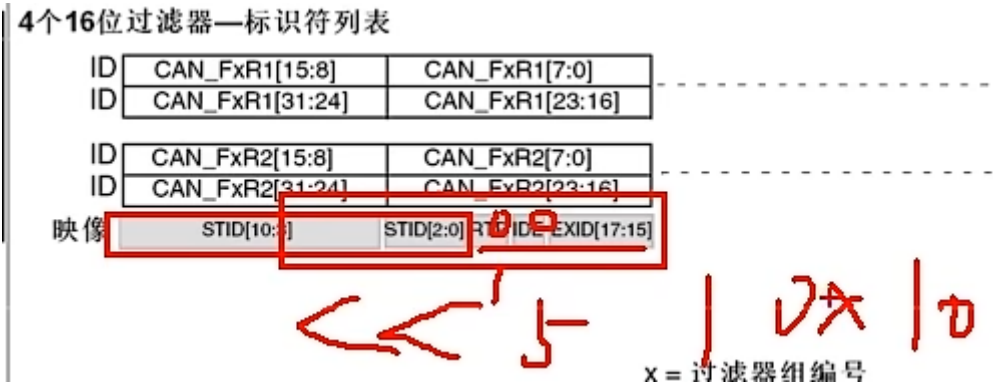
MASK：表明必须匹配id的那几位，这里给0x700左移五位，表示再左移五位表示1110 0000 0000 0000，即前三位要匹配

2 过滤器配置示例

总线上存在的ID	想要接收的ID	过滤器模式	R1[31:0]配置值	R2[31:0]配置值
0x123, 0x234, 0x345, 0x456, 0x567, 0x678	0x234, 0x345, 0x567	16位/列表	ID: R1[15:0]=0x234<<5 ID: R1[31:16]=0x345<<5	ID: R2[15:0]=0x567<<5 ID: R2[31:16]=0x000<<5
0x100~0x1FF, 0x200~0x2FF, 0x310~0x31F, 0x320~0x32F	0x200~0x2FF, 0x320~0x32F	16位/屏蔽	0x2是关键 ID: R1[15:0]=0x200<<5 Mask: R1[31:16]= (0x700<<5) 0x10 0x8	ID: R2[15:0]=0x320<<5 Mask: R2[31:16]= (0x7F0<<5) 0x10 0x8
0x123, 0x234, 0x345, 0x456, 0x12345678, 0x0789ABCD	0x123, 0x12345678	32位/列表	ID: R1[31:0]= 0x123<<21	ID: R2[31:0]= (0x12345678<<3) 0x4
0x12345600~0x123456FF, 0x0789AB00~0x0789ABFF	0x12345600~0x123456FF	32位/屏蔽	ID: R1[31:0]= (0x12345600<<3) 0x4	Mask: R2[31:0]= (0x1FFFFFF00<<3) 0x4 0x2
任意ID	只要遥控帧	32位/屏蔽	ID: R1[31:0]=0x2	Mask: R2[31:0]=0x2
任意ID	所有帧	32位/屏蔽	ID: R1[31:0]=随意	Mask: R2[31:0]=0

后面这种或是给IDE置1

这里为什么要左移<<5， 16位：id是11位，总共是16位，因为写入是右对齐，所以要左移16-11=5位
因为地址左移5位才可以是id的地址 32位：将11位id放到32位当中，最高位是31位，最低位是21位。故左移21位。



ID: R1[15:0]=0x200<<5//写入0x200左移5, 然后就是, RTR, IDE没有写入默认是0
Mask: R1[31:16]=
(0x700<<5) | 0x10 | 0x8

第二排, 屏蔽, 为什么是0x700, 700:111 0000 0000, 左移5位就是, 屏蔽, 高3位一样了。

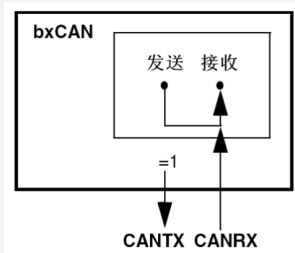
然后就是 | 0x10, (10:1000, 就是RTR要一样, 然后就是 | ide 0x8)

这里或上0x8是表示IDE位和上面的ID的IDE位保持一致, 注意这里屏蔽位置1都是跟上面一样, 列表一样, 置0就是随便, 注意这里IDE在16位就得一样, 防止错误

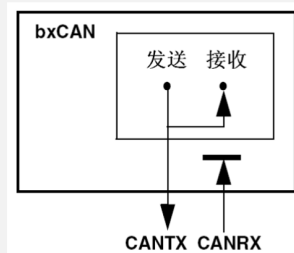
3 测试模式



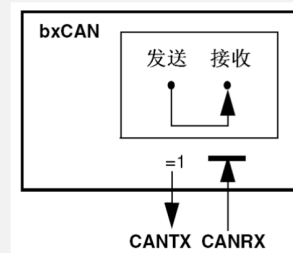
- 静默模式: 用于分析CAN总线的活动, 不会对总线造成影响
- 环回模式: 用于自测试, 同时发送的报文可以在CAN_TX引脚上检测到
- 环回静默模式: 用于热自测试, 自测的同时不会影响CAN总线



静默模式



环回模式



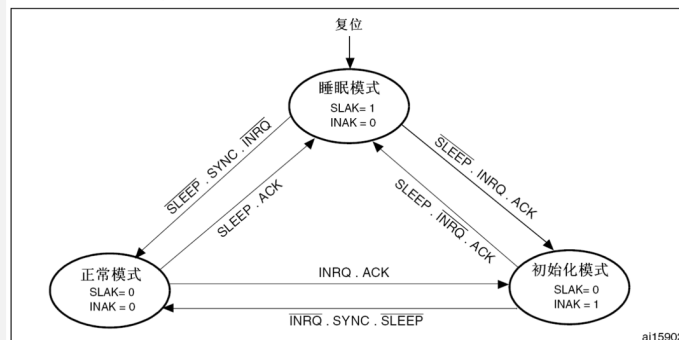
环回静默模式

3 工作模式



- 初始化模式: 用于配置CAN外设, 禁止报文的接收和发送
- 正常模式: 配置CAN外设后进入正常模式, 以便正常接收和发送报文
- 睡眠模式: 低功耗, CAN外设时钟停止, 可使用软件唤醒或者硬件自动唤醒
- AWUM: 置1, 自动唤醒, 一旦检测到CAN总线活动, 硬件就自动清零SLEEP, 唤醒CAN外设; 置0, 手动唤醒, 软件清零SLEEP, 唤醒CAN外设

图196 bxCAN工作模式



SLAK:睡眠确认状态位, =1表示确认进入睡眠状态位

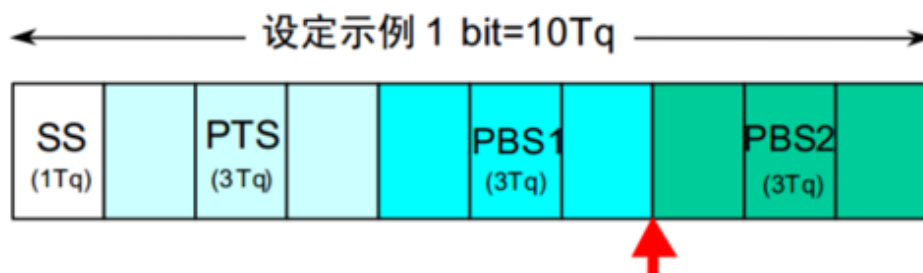
INAK:初始化确认位, =0表示未确认进入初始化模式

----不会马上进行, 还会等待一下, 就像下面的请求

INTRQ:请求进入初始化

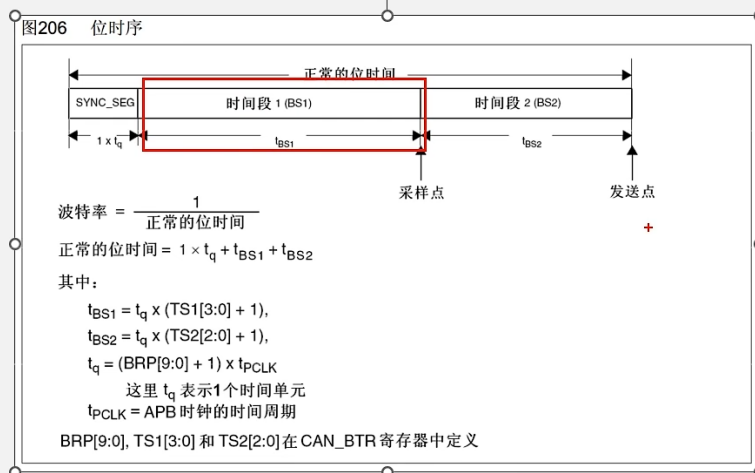
SLEEP:请求进入睡眠状态

SYNC:总线空闲



3 位时间特性

江协科技
jiangxiekeji.com



- SS = 1T_q
- BS1 = 1~16T_q
- BS2 = 1~8T_q
- SJW=1~4T_q

- 波特率 = APB1时钟频率 / 分频系数 / 一位的T_q数量
= 36MHz / (BRP[9:0]+1) / (1 + (TS1[3:0]+1) + (TS2[2:0]+1))

这里BS1把上面的PTS+PBS1合并起来了

3 中断

江协科技
jiangxiekeji.com

- CAN外设占用4个专用的中断向量
- **发送中断**: 发送邮箱空时产生
- **FIFO 0中断**: 收到一个报文/FIFO 0满/FIFO 0溢出时产生
- **FIFO 1中断**: 收到一个报文/FIFO 1满/FIFO 1溢出时产生
- **状态改变错误中断**: 出错/唤醒/进入睡眠时产生

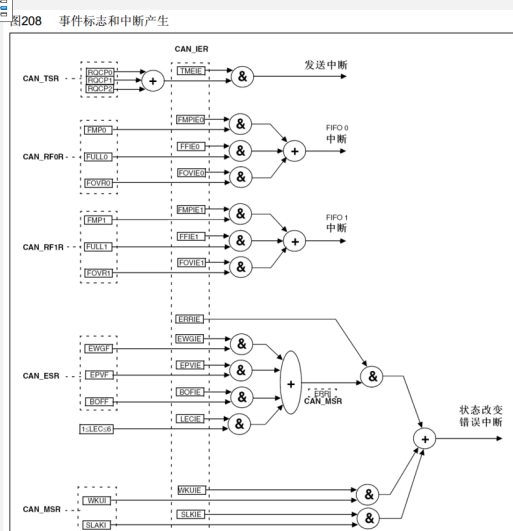
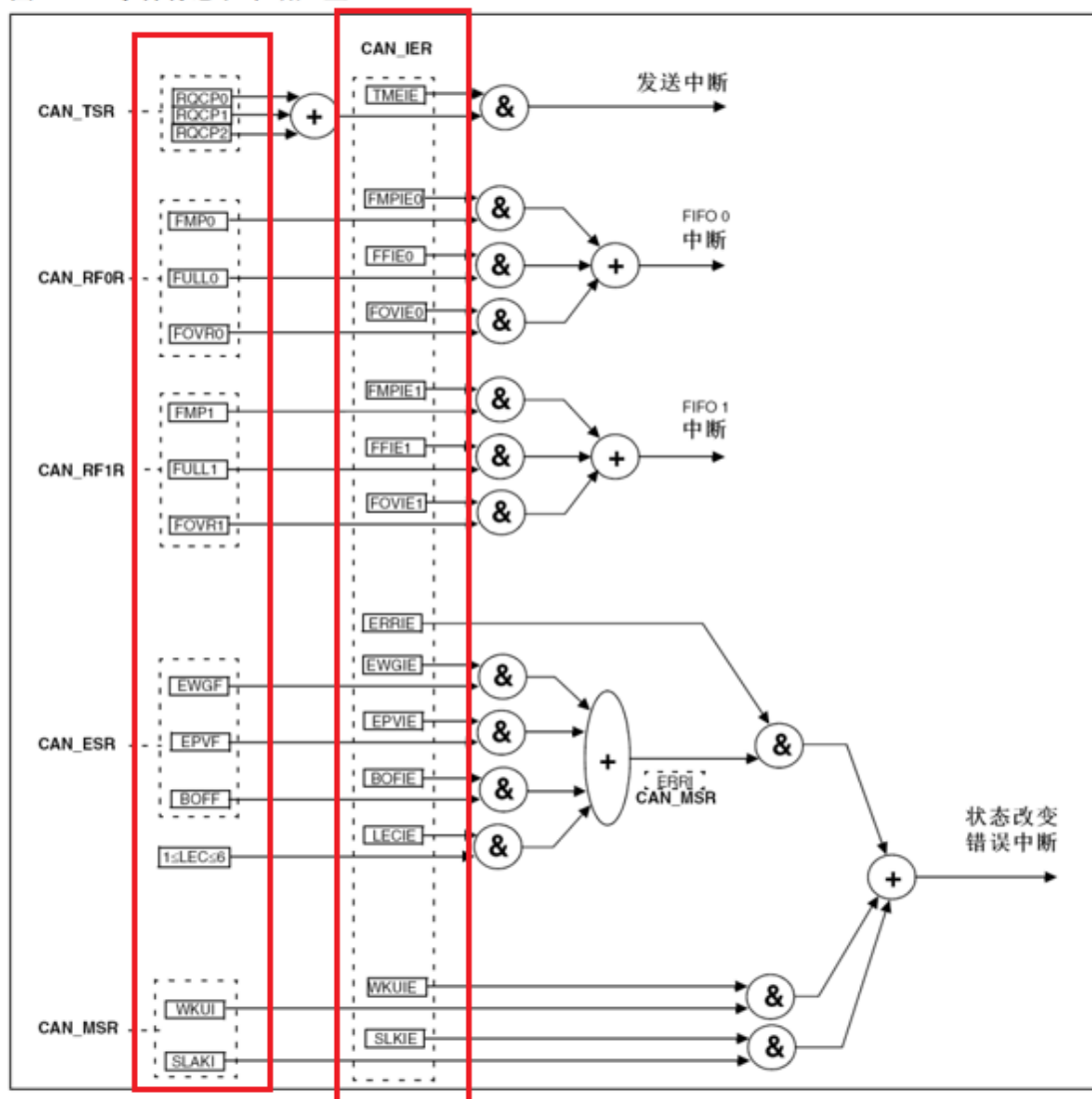


图208 事件标志和中断产生



左图,

最左边的一列, **中断信号源**

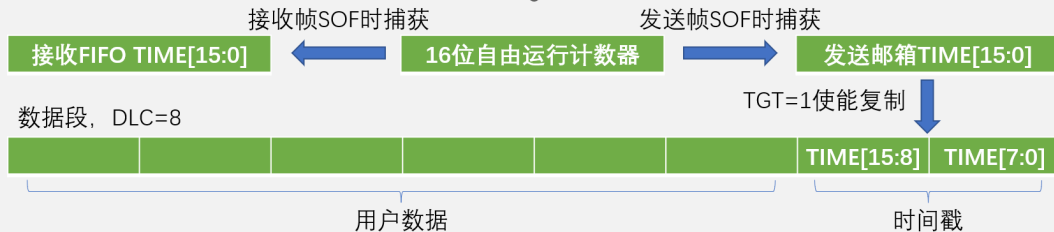
中间的一列是, **中断使能寄存器** (是否使能中断信号输出) ITConfig函数配置的就是这些IE寄存器

看手册, **CAN_IER寄存器**有具体说明

位3	FOVIE0: FIFO 0溢出中断使能 (FIFO overrun interrupt enable) 0: 当FIFO 0的FOVR位被置'1'时, 不产生中断; 1: 当FIFO 0的FOVR位被置'1'时, 产生中断。
位2	FFIE0: FIFO 0满中断使能 (FIFO full interrupt enable) 0: 当FIFO 0的FULL位被置'1'时, 不产生中断; 1: 当FIFO 0的FULL位被置'1'时, 产生中断。
位1	FMPIE0: FIFO 0消息挂号中断使能 (FIFO message pending interrupt enable) 0: 当FIFO 0的FMP[1:0]位为非0时, 不产生中断; 1: 当FIFO 0的FMP[1:0]位为非0时, 产生中断。
位0	TMEIE: 发送邮箱空中断使能 (Transmit mailbox empty interrupt enable) 0: 当RQCPx位被置'1'时, 不产生中断; 1: 当RQCPx位被置'1'时, 产生中断。 注: 请参考22.8节bxCAN中断。

3 时间触发通信

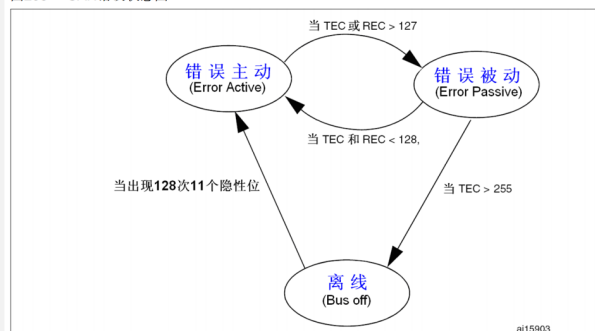
- **TTCM**: 置1, 开启时间触发通信功能; 置0, 关闭时间触发通信功能
- CAN外设内置一个**16位的计数器**, 用于记录时间戳
- TTCM置1后, 该计数器在每个CAN位的时间自增一次, 溢出后归零
- 每个**发送邮箱和接收FIFO**都有一个**TIME[15:0]寄存器**, 发送帧SOF时, 硬件捕获计数器值到发送邮箱的TIME寄存器, 接收帧SOF时, 硬件捕获计数器值到接收FIFO的TIME寄存器
- 发送邮箱可配置TGT位, 捕获计数器值的同时, 也把此值写入到数据帧数据段的最后两个字节, 为了使用此功能, DLC必须设置为8

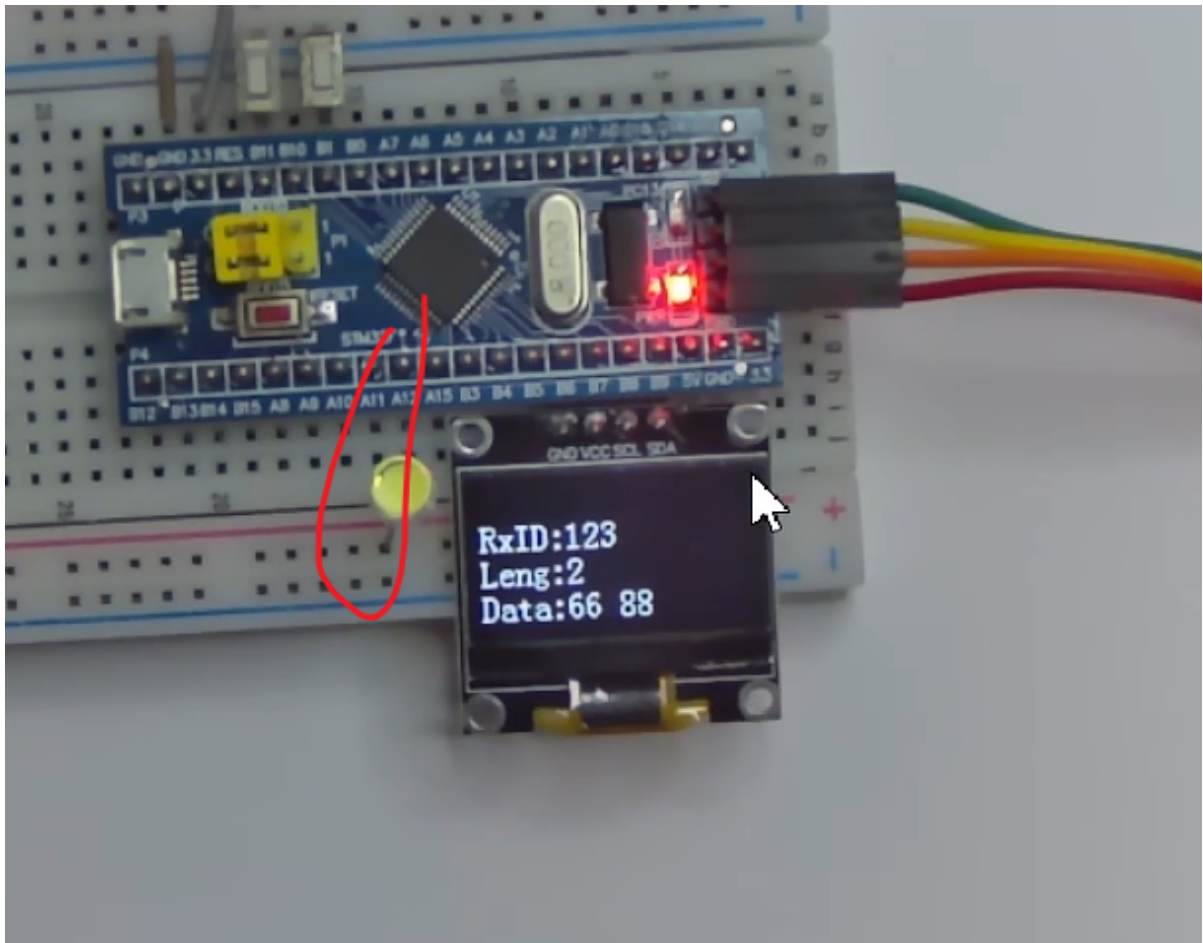


3 错误处理和离线恢复

- TEC和REC根据错误的情况增加或减少
- **ABOM**: 置1, 开启离线自动恢复, 进入离线状态后, 就自动开启恢复过程; 置0, 关闭离线自动恢复, 软件必须先请求进入然后再退出初始化模式, 随后恢复过程才被开启

图205 CAN错误状态图





环回模式调试：可以插个led，按下按键闪烁，说明有波形发出，就说明发送没有问题，也可以通过此方法，判断代码的问题，是发送的问题还是接收的问题

1.单个设备的回环测试和三个设备互相通信

```
#include "stm32f10x.h"                                // Device header

void MyCAN_Init(void)
{
    RCC_APB2PeriphClockCmd(RCC_APB2Periph_GPIOA, ENABLE);    开GPIOA时钟
    RCC_APB1PeriphClockCmd(RCC_APB1Periph_CAN1, ENABLE);    开CAN外设时钟，它挂载在APB1上。

    GPIO_InitTypeDef GPIO_InitStructure;
    GPIO_InitStructure.GPIO_Mode = GPIO_Mode_AF_PP;
    GPIO_InitStructure.GPIO_Pin = GPIO_Pin_12;
    GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;
    GPIO_Init(GPIOA, &GPIO_InitStructure);

    GPIO_InitStructure.GPIO_Mode = GPIO_Mode_IPU;
    GPIO_InitStructure.GPIO_Pin = GPIO_Pin_11;
    GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;
    GPIO_Init(GPIOA, &GPIO_InitStructure);

    CAN_InitTypeDef CAN_InitStructure;
    CAN_InitStructure.CAN_Mode = CAN_Mode_LoopBack;
```

初始化GPIOA，配置的是PA12和PA11两个引脚

这里要用复用模式，引脚控制权是给CAN外设的

选择上拉输出模式，因为引脚默认高电平

```

//这里是环回模式，如果是多个设备的话，那就是改为normal正常模式就好了
//这个模式mode是测试模式，不是工作模式，就是正常，静默，环回和环回 静默
CAN_InitStructure.CAN_Prescaler = 48; //波特率 = 36M / 48 / (1 + 2 + 3)
= 125k
CAN_InitStructure.CAN_BS1 = CAN_BS1_2tq; //高速CAN是125k-1M波特率
CAN_InitStructure.CAN_BS2 = CAN_BS2_3tq; //也不用手动-1
CAN_InitStructure.CAN_SJW = CAN_SJW_2tq; 再补偿宽度
CAN_InitStructure.CAN_NART = DISABLE; //自动重传 disable-0
CAN_InitStructure.CAN_TXFP = DISABLE; //优先级由报文标识符决定，标识符小的先发送
CAN_InitStructure.CAN_RFLM = DISABLE; //禁用FIFO, 报文溢出时，被新报文覆盖
CAN_InitStructure.CAN_AWUM = DISABLE; //手动唤醒外设
CAN_InitStructure.CAN_TTCM = DISABLE; //关闭时间触发通信功能，就是时间戳
CAN_InitStructure.CAN_ABOM = DISABLE; //关闭离线自动恢复
CAN_Init(CAN1, &CAN_InitStructure);

CAN_FilterInitTypeDef CAN_FilterInitStructure;
CAN_FilterInitStructure.CAN_FilterNumber = 0; //过滤器0-13
CAN_FilterInitStructure.CAN_FilterIdHigh = 0x0000; //下面4个就是r1,r2的高低位，比
如32位屏蔽，就是前两个存入32位id,屏蔽位存入后两个
CAN_FilterInitStructure.CAN_FilterIdLow = 0x0000;
CAN_FilterInitStructure.CAN_FilterMaskIdHigh = 0x0000;
CAN_FilterInitStructure.CAN_FilterMaskIdLow = 0x0000;
CAN_FilterInitStructure.CAN_FilterScale = CAN_FilterScale_32bit; //选择过滤器位
宽，32还是16
CAN_FilterInitStructure.CAN_FilterMode = CAN_FilterMode_IdMask; //过滤模式，这
里是屏蔽模式
CAN_FilterInitStructure.CAN_FilterFIFOAssignment = CAN_Filter_FIFO0; //进
fifo0,还是fifi1
CAN_FilterInitStructure.CAN_FilterActivation = ENABLE; //激活过滤器
CAN_FilterInit(&CAN_FilterInitStructure); 过滤器配置
}

```

不用手动减一因为写入时候自动帮减

```

void MyCAN_Transmit(uint32_t ID, uint8_t Length, uint8_t *Data) //这里是data是指针，
因为数组传进函数，可以用指针接收，更高效率
{
    CanTxMsg TxMessage;
    TxMessage.StdId = ID;
    TxMessage.ExtId = ID; //扩展id
    TxMessage.IDE = CAN_Id_Standard; //CAN_ID_STD
    TxMessage.RTR = CAN_RTR_Data; //遥控帧，遥控帧标志位
    TxMessage.DLC = Length; //数据段长度
    for (uint8_t i = 0; i < Length; i++)
    {
        TxMessage.Data[i] = Data[i];
    }

    uint8_t TransmitMailbox = CAN_Transmit(CAN1, &TxMessage); //返回值是一个邮箱
    //请求过去不会马上开始传输，要等待一会的

    uint32_t Timeout = 0;
    while (CAN_TransmitStatus(CAN1, TransmitMailbox) != CAN_TxStatus_Ok) //所以这里
    等待发送完成
    {
        Timeout++;
        if (Timeout > 100000) //这里是等待超时退出
    }
}

```

```

    {
        break;
    }
}
}

uint8_t MyCAN_ReceiveFlag(void)
{
    if (CAN_MessagePending(CAN1, CAN_FIFO0) > 0) //fifo里面报文排队的个数
    {
        return 1;
    }
    return 0;
}

//这里使用的是查询方式接收，还有一种是中断的形式接收

void MyCAN_Receive(uint32_t *ID, uint8_t *Length, uint8_t *Data)
    //输出参数，因为函数不支持返回多个值，所以用指针就比较好
{
    CanRxMsg RxMessage;
    CAN_Receive(CAN1, CAN_FIFO0, &RxMessage);

    if (RxMessage.IDE == CAN_Id_Standard)
    {
        *ID = RxMessage.StdId;    是标准格式，就把标准id格式赋值给*ID
    }
    else
    {
        *ID = RxMessage.ExtId;
    }

    if (RxMessage.RTR == CAN_RTR_Data)
    {
        *Length = RxMessage.DLC;
        for (uint8_t i = 0; i < *Length; i++)    表示当前收到的是数据帧，赋值data
        {
            Data[i] = RxMessage.Data[i];
        }
    }
    else
    {
        //...
    }
}

```

```

#include "stm32f10x.h"           // Device header
#include "Delay.h"
#include "OLED.h"
#include "Key.h"
#include "MyCAN.h"

uint8_t KeyNum;
uint32_t TxID = 0x555;

```

```

uint8_t TxLength = 4;
uint8_t TxData[8] = {0x11, 0x22, 0x33, 0x44};

uint32_t RxID;
uint8_t RxLength;
uint8_t RxData[8];

int main(void)
{
    OLED_Init();
    Key_Init();
    MyCAN_Init();

    OLED_ShowString(1, 1, "TxID:");
    OLED_ShowHexNum(1, 6, TxID, 3);
    OLED_ShowString(2, 1, "RxID:");
    OLED_ShowString(3, 1, "Leng:");
    OLED_ShowString(4, 1, "Data:");

    while (1)
    {
        KeyNum = Key_GetNum();

        if (KeyNum == 1)
        {
            TxData[0] ++;
            TxData[1] ++;
            TxData[2] ++;
            TxData[3] ++;

            MyCAN_Transmit(TxID, TxLength, TxData);
        }

        if (MyCAN_ReceiveFlag())
        {
            MyCAN_Receive(&RxID, &RxLength, RxData); //数组名就是地址，所以不用取地址符

            OLED_ShowHexNum(2, 6, RxID, 3); //记得用十六进制显示
            OLED_ShowHexNum(3, 6, RxLength, 1);
            OLED_ShowHexNum(4, 6, RxData[0], 2);
            OLED_ShowHexNum(4, 9, RxData[1], 2);
            OLED_ShowHexNum(4, 12, RxData[2], 2);
            OLED_ShowHexNum(4, 15, RxData[3], 2);
        }
    }
}

```

2.标准格式-扩展格式-数据帧-遥控帧（查询接收）

结构体赋值，可以用上面的，也可以用下面的.xx,.xx,这里前提都是CanTxMsg类型的哈


```

13 CanTxMsg TxMsg = {
14     /* StdId      ExtId      IDE      RTR      DLC      Data[8]      */
15     0x555, 0x00000000, CAN_Id_Standard, CAN_RTR_Data, 4, {0x11, 0x22, 0x33, 0x44}
16 };
17
18 CanTxMsg a = {
19     .StdId = 0x555,
20     .ExtId = 0x00000000,
21     .DLC = 4
22 };
23
24

```

主函数中用结构体形式传递参数，结构体定义这一块在配置文件里已经规定好了。

←的是指定成员名赋值，上下两种办法都可以的。

```

void MyCAN_Transmit(CanTxMsg *TxMessage) //这里是直接用CAN这个结构体传参，如上图所示，结构体指针传参
{
    uint8_t TransmitMailbox = CAN_Transmit(CAN1, TxMessage);

    uint32_t Timeout = 0;
    while (CAN_TransmitStatus(CAN1, TransmitMailbox) != CAN_TxStatus_Ok)
    {
        Timeout ++;
        if (Timeout > 100000)
        {
            break;
        }
    }
}

```

```

void MyCAN_Receive(CanRxMsg *RxMessage) //接收也是这个结构体，省略了
{
    CAN_Receive(CAN1, CAN_FIFO0, RxMessage);
}

```

```

CanTxMsg TxMsgArray[] = {
    /* StdId      ExtId      IDE      RTR      DLC      Data[8]      */
    /*
    {0x555, 0x00000000, CAN_Id_Standard, CAN_RTR_Data, 4, {0x11, 0x22, 0x33, 0x44}},
    {0x000, 0x12345678, CAN_Id_Extended, CAN_RTR_Data, 4, {0xAA, 0xBB, 0xCC, 0xDD}},
    {0x666, 0x00000000, CAN_Id_Standard, CAN_RTR_Remote, 0, {0x00, 0x00, 0x00, 0x00}},
    {0x000, 0x0789ABCD, CAN_Id_Extended, CAN_RTR_Remote, 0, {0x00, 0x00, 0x00, 0x00}},
    */
}; //这个数组类型是CanTxMsg

uint8_t pTxMsgArray = 0;

CanRxMsg RxMsg;

```

注意这里是必须CanRxMsg类型的结构体类型

```

while (1)
{
    KeyNum = Key_GetNum();
}

```

```

if (KeyNum == 1)//发送
{
    MyCAN_Transmit(&TxMsgArray[pTxMsgArray]);

    pTxMsgArray ++;
    if (pTxMsgArray >= sizeof(TxMsgArray) / sizeof(CanTxMsg))
    {
        pTxMsgArray = 0;
    }
}

if (MyCAN_ReceiveFlag())//接收
{
    MyCAN_Receive(&RxMsg);

    if (RxMsg.IDE == CAN_Id_Standard)
    {
        OLED_ShowString(1, 6, "Std");

        OLED_ShowHexNum(2, 6, RxMsg.StdId, 8);
    }
    else if (RxMsg.IDE == CAN_Id_Extended)
    {
        OLED_ShowString(1, 6, "Ext");

        OLED_ShowHexNum(2, 6, RxMsg.ExtId, 8);
    }

    if (RxMsg.RTR == CAN_RTR_Data)
    {
        OLED_ShowString(1, 10, "Data ");

        OLED_ShowHexNum(3, 6, RxMsg.DLC, 1);

        OLED_ShowHexNum(4, 6, RxMsg.Data[0], 2);
        OLED_ShowHexNum(4, 9, RxMsg.Data[1], 2);
        OLED_ShowHexNum(4, 12, RxMsg.Data[2], 2);
        OLED_ShowHexNum(4, 15, RxMsg.Data[3], 2);
    }
    else if (RxMsg.RTR == CAN_RTR_Remote)
    {
        OLED_ShowString(1, 10, "Remote");

        OLED_ShowHexNum(3, 6, RxMsg.DLC, 1);

        OLED_ShowHexNum(4, 6, 0x00, 2);
        OLED_ShowHexNum(4, 9, 0x00, 2);
        OLED_ShowHexNum(4, 12, 0x00, 2);
        OLED_ShowHexNum(4, 15, 0x00, 2);
    }
}
}

```

同一个总线当中处于正常模式的设备应当有两个，这样才有发送应答。

取第p项发送，p初始是0，p发送一次自增一次。
这里除的意思是计算元素个数

keil里按住alt可以框选..

3.过滤

如果这里要过滤遥控帧，那就是要加上RTR，



```
CAN_FilterInitStructure.CAN_FilterIdHigh = (0x234 << 5)|0x10;
```

16为列表模式:

```
CAN_FilterInitTypeDef CAN_FilterInitStructure;
CAN_FilterInitStructure.CAN_FilterNumber = 0;//第0个过滤器
CAN_FilterInitStructure.CAN_FilterIdHigh = 0x234 << 5;//这里是16位的列表的过滤，
过滤2234,345,567, , ,
CAN_FilterInitStructure.CAN_FilterIdLow = 0x345 << 5;
CAN_FilterInitStructure.CAN_FilterMaskIdHigh = 0x567 << 5;
CAN_FilterInitStructure.CAN_FilterMaskIdLow = 0x000 << 5;
CAN_FilterInitStructure.CAN_FilterScale = CAN_FilterScale_16bit;
CAN_FilterInitStructure.CAN_FilterMode = CAN_FilterMode_IdList;
CAN_FilterInitStructure.CAN_FilterFIFOAssignment = CAN_Filter_FIFO0;
CAN_FilterInitStructure.CAN_FilterActivation = ENABLE;
CAN_FilterInit(&CAN_FilterInitStructure);
```

```
CanTxMsg TxMsgArray[] = {
/*      StdId      ExtId      IDE      RTR      DLC      Data[8]
*/
    {0x123, 0x00000000, CAN_Id_Standard, CAN_RTR_Data, 4, {0x11, 0x22, 0x33, 0x44}},
    {0x234, 0x00000000, CAN_Id_Standard, CAN_RTR_Remote, 4, {0x11, 0x22, 0x33, 0x44}},
    {0x345, 0x00000000, CAN_Id_Standard, CAN_RTR_Data, 4, {0x11, 0x22, 0x33, 0x44}},
    {0x456, 0x00000000, CAN_Id_Standard, CAN_RTR_Data, 4, {0x11, 0x22, 0x33, 0x44}},
    {0x567, 0x00000000, CAN_Id_Standard, CAN_RTR_Data, 4, {0x11, 0x22, 0x33, 0x44}},
    {0x678, 0x00000000, CAN_Id_Standard, CAN_RTR_Data, 4, {0x11, 0x22, 0x33, 0x44}},
};
```

16位屏蔽模式

```
CAN_FilterInitTypeDef CAN_FilterInitStructure;
CAN_FilterInitStructure.CAN_FilterNumber = 0;

CAN_FilterInitStructure.CAN_FilterIdHigh = 0x200 << 5; //屏蔽过滤200组
CAN_FilterInitStructure.CAN_FilterMaskIdHigh = (0x700 << 5) | 0x10 | 0x8; //这
里是RTR和IDE为1，就是和上面保持一致

CAN_FilterInitStructure.CAN_FilterIdLow = 0x320 << 5; //屏蔽过滤320组，320是11
0010 0000
CAN_FilterInitStructure.CAN_FilterMaskIdLow = (0x7F0 << 5) | 0x10 | 0x8; //7f0
是111 1111 0000

CAN_FilterInitStructure.CAN_FilterScale = CAN_FilterScale_16bit;
CAN_FilterInitStructure.CAN_FilterMode = CAN_FilterMode_IdMask;
CAN_FilterInitStructure.CAN_FilterFIFOAssignment = CAN_Filter_FIFO0;
CAN_FilterInitStructure.CAN_FilterActivation = ENABLE;
CAN_FilterInit(&CAN_FilterInitStructure);
```

```
CanTxMsg TxMsgArray[] = {
/*      StdId      ExtId      IDE      RTR      DLC      Data[8]
*/
    {0x100, 0x00000000, CAN_Id_Standard, CAN_RTR_Data, 4, {0x11, 0x22, 0x33,
0x44}},
    {0x101, 0x00000000, CAN_Id_Standard, CAN_RTR_Data, 4, {0x11, 0x22, 0x33,
0x44}},
    {0x1FE, 0x00000000, CAN_Id_Standard, CAN_RTR_Data, 4, {0x11, 0x22, 0x33,
0x44}},
    {0x1FF, 0x00000000, CAN_Id_Standard, CAN_RTR_Data, 4, {0x11, 0x22, 0x33,
0x44}},

    {0x200, 0x00000000, CAN_Id_Standard, CAN_RTR_Data, 4, {0x11, 0x22, 0x33,
0x44}},
    {0x201, 0x00000000, CAN_Id_Standard, CAN_RTR_Data, 4, {0x11, 0x22, 0x33,
0x44}},
    {0x2FE, 0x00000000, CAN_Id_Standard, CAN_RTR_Data, 4, {0x11, 0x22, 0x33,
0x44}},
    {0x2FF, 0x00000000, CAN_Id_Standard, CAN_RTR_Data, 4, {0x11, 0x22, 0x33,
0x44}},

    {0x310, 0x00000000, CAN_Id_Standard, CAN_RTR_Data, 4, {0x11, 0x22, 0x33,
0x44}},
    {0x311, 0x00000000, CAN_Id_Standard, CAN_RTR_Data, 4, {0x11, 0x22, 0x33,
0x44}},
    {0x31E, 0x00000000, CAN_Id_Standard, CAN_RTR_Data, 4, {0x11, 0x22, 0x33,
0x44}},
    {0x31F, 0x00000000, CAN_Id_Standard, CAN_RTR_Data, 4, {0x11, 0x22, 0x33,
0x44}},

    {0x320, 0x00000000, CAN_Id_Standard, CAN_RTR_Data, 4, {0x11, 0x22, 0x33,
0x44}},
    {0x321, 0x00000000, CAN_Id_Standard, CAN_RTR_Data, 4, {0x11, 0x22, 0x33,
0x44}},
```

```

        {0x32E, 0x00000000, CAN_Id_Standard, CAN_RTR_Data, 4, {0x11, 0x22, 0x33, 0x44}},
        {0x32F, 0x00000000, CAN_Id_Standard, CAN_RTR_Data, 4, {0x11, 0x22, 0x33, 0x44}},
    };

```

32位列表模式

扩展格式是29位，标准格式是11位，`uint32_t ID2 = (0x12345678u << 3) | 0x4;`超过了就会没用，硬件会自动舍去

```

CAN_FilterInitTypeDef CAN_FilterInitStructure;
CAN_FilterInitStructure.CAN_FilterNumber = 0;

uint32_t ID1 = 0x123 << 21; //这里把32位左移21位 0001 0010 0011 → 0001 0010 0011 0000 0000 0000 0000 0000 就是0x9180 0000
CAN_FilterInitStructure.CAN_FilterIdHigh = ID1 >> 16; //取高16位, 0x0180
CAN_FilterInitStructure.CAN_FilterIdLow = ID1; //取低16位, 0x0000

uint32_t ID2 = (0x12345678u << 3) | 0x4; //这里转为无符号的, 不然会超出界限了, +u
CAN_FilterInitStructure.CAN_FilterMaskIdHigh = ID2 >> 16;
CAN_FilterInitStructure.CAN_FilterMaskIdLow = ID2; //还可以加上&0xffff

CAN_FilterInitStructure.CAN_FilterScale = CAN_FilterScale_32bit;
CAN_FilterInitStructure.CAN_FilterMode = CAN_FilterMode_IdList;
CAN_FilterInitStructure.CAN_FilterFIFOAssignment = CAN_Filter_FIFO0;
CAN_FilterInitStructure.CAN_FilterActivation = ENABLE;
CAN_FilterInit(&CAN_FilterInitStructure);

```

```

CanTxMsg TxMsgArray[] = {
    /*      StdId      ExtId      IDE      RTR      DLC      Data[8]
    */
    {0x123, 0x00000000, CAN_Id_Standard, CAN_RTR_Data, 4, {0x11, 0x22, 0x33, 0x44}},
    {0x234, 0x00000000, CAN_Id_Standard, CAN_RTR_Data, 4, {0x11, 0x22, 0x33, 0x44}},
    {0x345, 0x00000000, CAN_Id_Standard, CAN_RTR_Data, 4, {0x11, 0x22, 0x33, 0x44}},
    {0x456, 0x00000000, CAN_Id_Standard, CAN_RTR_Data, 4, {0x11, 0x22, 0x33, 0x44}},

    {0x000, 0x12345678, CAN_Id_Extended, CAN_RTR_Data, 4, {0x11, 0x22, 0x33, 0x44}}, //扩展格式下, 标准id无效, 所以给000, 防止出错
    {0x000, 0x0789ABCD, CAN_Id_Extended, CAN_RTR_Data, 4, {0x11, 0x22, 0x33, 0x44}},
};

```

如果想一个查看全部，一个是过滤的数据，那么查看全部的那个就要是32位屏蔽你模式，也要是环回模式

`CAN_InitStructure.CAN_Mode = CAN_Mode_LoopBack;`

CAN_FilterInitStructure.CAN_FilterMode = CAN_FilterMode_IdMask;

任意ID	只要遥控帧	32位/屏蔽	ID: R1[31:0]=0x2	Mask: R2[31:0]=0x2
任意ID	所有帧	32位/屏蔽	ID: R1[31:0]=随意	Mask: R2[31:0]=0

```
CAN_FilterInitStructure.CAN_FilterIdHigh = 0x0000; //其他都选择000 9
CAN_FilterInitStructure.CAN_FilterIdLow = 0x0000;

CAN_FilterInitStructure.CAN_FilterMaskIdHigh = 0x0000;
CAN_FilterInitStructure.CAN_FilterMaskIdLow = 0x0000;

CAN_FilterInitStructure.CAN_FilterScale = CAN_FilterScale_32bit;
CAN_FilterInitStructure.CAN_FilterMode = CAN_FilterMode_IdMask; //这里
CAN_FilterInitStructure.CAN_FilterFIFOAssignment = CAN_Filter_FIFO0;
CAN_FilterInitStructure.CAN_FilterActivation = ENABLE;
CAN_FilterInit(&CAN_FilterInitStructure);
```

**32位屏蔽模式

```
CanTxMsg TxMsgArray[] = {
/*   StdId      ExtId      IDE      RTR      DLC      Data[8]
*/
    {0x000, 0x12345600, CAN_Id_Extended, CAN_RTR_Data, 4, {0x11, 0x22, 0x33, 0x44}},
    {0x000, 0x12345601, CAN_Id_Extended, CAN_RTR_Data, 4, {0x11, 0x22, 0x33, 0x44}},
    {0x000, 0x123456FE, CAN_Id_Extended, CAN_RTR_Data, 4, {0x11, 0x22, 0x33, 0x44}},
    {0x000, 0x123456FF, CAN_Id_Extended, CAN_RTR_Data, 4, {0x11, 0x22, 0x33, 0x44}},

    {0x000, 0x0789AB00, CAN_Id_Extended, CAN_RTR_Data, 4, {0x11, 0x22, 0x33, 0x44}},
    {0x000, 0x0789AB01, CAN_Id_Extended, CAN_RTR_Data, 4, {0x11, 0x22, 0x33, 0x44}},
    {0x000, 0x0789ABFE, CAN_Id_Extended, CAN_RTR_Data, 4, {0x11, 0x22, 0x33, 0x44}},
    {0x000, 0x0789ABFF, CAN_Id_Extended, CAN_RTR_Data, 4, {0x11, 0x22, 0x33, 0x44}},
};
```

```
CAN_FilterInitTypeDef CAN_FilterInitStructure;
CAN_FilterInitStructure.CAN_FilterNumber = 0;

uint32_t ID = (0x12345600u << 3) | 0x4;
CAN_FilterInitStructure.CAN_FilterIdHigh = ID >> 16;
CAN_FilterInitStructure.CAN_FilterIdLow = ID;

uint32_t Mask = (0x1FFFFFF00u << 3) | 0x4 | 0x2;
CAN_FilterInitStructure.CAN_FilterMaskIdHigh = Mask >> 16;
```

```

CAN_FilterInitStructure.CAN_FilterMaskIdLow = Mask;

CAN_FilterInitStructure.CAN_FilterScale = CAN_FilterScale_32bit;
CAN_FilterInitStructure.CAN_FilterMode = CAN_FilterMode_IdMask;
CAN_FilterInitStructure.CAN_FilterFIFOAssignment = CAN_Filter_FIFO0;
CAN_FilterInitStructure.CAN_FilterActivation = ENABLE;
CAN_FilterInit(&CAN_FilterInitStructure);
---
```

只要遥控帧

uint32_t ID = 0x2;

要是只要数据帧的话， uint32_t ID = 0x0;

```

CAN_FilterInitTypeDef CAN_FilterInitStructure;
CAN_FilterInitStructure.CAN_FilterNumber = 0;

uint32_t ID = 0x2;
CAN_FilterInitStructure.CAN_FilterIdHigh = ID >> 16;
CAN_FilterInitStructure.CAN_FilterIdLow = ID;

uint32_t Mask = 0x2; //必须跟上面一样匹配，注意这里只是是否相同，1表示必须相同，0表示随意
CAN_FilterInitStructure.CAN_FilterMaskIdHigh = Mask >> 16;
CAN_FilterInitStructure.CAN_FilterMaskIdLow = Mask;

CAN_FilterInitStructure.CAN_FilterScale = CAN_FilterScale_32bit;
CAN_FilterInitStructure.CAN_FilterMode = CAN_FilterMode_IdMask;
CAN_FilterInitStructure.CAN_FilterFIFOAssignment = CAN_Filter_FIFO0;
CAN_FilterInitStructure.CAN_FilterActivation = ENABLE;
CAN_FilterInit(&CAN_FilterInitStructure);
```

```

CanTxMsg TxMsgArray[] = {
/*      StdId      ExtId      IDE      RTR      DLC      Data[8]
*/
    {0x123, 0x00000000, CAN_Id_Standard, CAN_RTR_Data, 4, {0x11, 0x22, 0x33, 0x44}},
    {0x234, 0x00000000, CAN_Id_Standard, CAN_RTR_Remote, 0, {0x00, 0x00, 0x00, 0x00}},
    {0x345, 0x00000000, CAN_Id_Standard, CAN_RTR_Data, 4, {0x11, 0x22, 0x33, 0x44}},
    {0x456, 0x00000000, CAN_Id_Standard, CAN_RTR_Remote, 0, {0x00, 0x00, 0x00, 0x00}},

    {0x000, 0x12345600, CAN_Id_Extended, CAN_RTR_Data, 4, {0x11, 0x22, 0x33, 0x44}},
    {0x000, 0x12345601, CAN_Id_Extended, CAN_RTR_Remote, 0, {0x00, 0x00, 0x00, 0x00}},
    {0x000, 0x123456FE, CAN_Id_Extended, CAN_RTR_Data, 4, {0x11, 0x22, 0x33, 0x44}},
}
```



```

        {0x000, 0x123456FF, CAN_Id_Extended, CAN_RTR_Remote, 0, {0x00, 0x00, 0x00,
0x00}},

        {0x000, 0x0789AB00, CAN_Id_Extended, CAN_RTR_Data, 4, {0x11, 0x22, 0x33,
0x44}},
        {0x000, 0x0789AB01, CAN_Id_Extended, CAN_RTR_Remote, 0, {0x00, 0x00, 0x00,
0x00}},
        {0x000, 0x0789ABFE, CAN_Id_Extended, CAN_RTR_Data, 4, {0x11, 0x22, 0x33,
0x44}},
        {0x000, 0x0789ABFF, CAN_Id_Extended, CAN_RTR_Remote, 0, {0x00, 0x00, 0x00,
0x00}},
};

```

9.中断式接收

单个帧用查询接收就好啦，

一个数据帧最大可以有8个字节，然后如果有更多的数据，那就要分开多帧，然后这里就是，可以现在中断里存入一个字节暂存，然后等存满了8个字节再置标志位，让主程序处理

CAN_IER寄存器的具体说明

位3	FOVIE0: FIFO 0溢出中断使能 (FIFO overrun interrupt enable) 0: 当FIFO 0的FOVR位被置'1'时，不产生中断； 1: 当FIFO 0的FOVR位被置'1'时，产生中断。
位2	FFIE0: FIFO 0满中断使能 (FIFO full interrupt enable) 0: 当FIFO 0的FULL位被置'1'时，不产生中断； 1: 当FIFO 0的FULL位被置'1'时，产生中断。
位1	FMPIE0: FIFO 0消息挂号中断使能 (FIFO message pending interrupt enable) 0: 当FIFO 0的FMP[1:0]位为非0时，不产生中断； 1: 当FIFO 0的FMP[1:0]位为非0时，产生中断。
位0	TMEIE: 发送邮箱空中断使能 (Transmit mailbox empty interrupt enable) 0: 当RQCPx位被置'1'时，不产生中断； 1: 当RQCPx位被置'1'时，产生中断。 注: 请参考22.8节bxCAN中断。

```

CanRxMsg MyCAN_RxMsg;
uint8_t MyCAN_RxFlag;

//开始中断配置
CAN_ITConfig(CAN1, CAN_IT_FMP0, ENABLE); //打开外设中断信号的输出, CAN_IT_FMP0是
fifo有报文时触发中断

NVIC_PriorityGroupConfig(NVIC_PriorityGroup_2);

NVIC_InitTypeDef NVIC_InitStructure; //配置nvic
NVIC_InitStructure.NVIC_IRQChannel = USB_LP_CAN1_RX0_IRQn;
//USB_HP_CAN1_TX_IRQn      发送中断
// USB_LP_CAN1_RX0_IRQn    接收fifo0中断
// CAN1_RX1_IRQn           接收fifo1中断
// CAN1_SCE_IRQn           状态改变和错误中断

```

```

NVIC_InitStructure.NVIC_IRQChannelCmd = ENABLE;
NVIC_InitStructure.NVIC_IRQChannelPreemptionPriority = 1;
NVIC_InitStructure.NVIC_IRQChannelSubPriority = 1;
NVIC_Init(&NVIC_InitStructure); //收到报文后就会跳转到中断函数

---

void USB_LP_CAN1_RX0_IRQHandler(void)
{
    if (CAN_GetITStatus(CAN1, CAN_IT_FMP0) == SET) //注意这里有2bit(因为要最大存3条数据呀), 即0,1,2,3然后就是, 大于0就表示有数据了

    {
        CAN_Receive(CAN1, CAN_FIFO0, &MyCAN_RxMsg); //接收到了就存到MyCAN_RxMsg结构变量里
        MyCAN_RxFlag = 1;
    }
    //不用再调用清零标志位CAN_CleanITPendingBit, 因为CAN_Receive最后会自动释放邮箱, 然后还有FMP0是2bit, 特殊不能直接清零
}

```

对比其他CAN中断标志

//需手动清除的标志:

//例如 CAN_IT_TME (发送邮箱空)、CAN_IT_FF0 (FIFO 0 满) 等, 需调用 CAN_ClearITPendingBit()。

//无需手动清除的标志:

//CAN_IT_FMP0 和 CAN_IT_FMP1 (FIFO 待处理消息) 由硬件自动管理。

```

mian:

if (MyCAN_RxFlag == 1) //中断标志收到了就是1
{
    MyCAN_RxFlag = 0; //然后再清零标志位

    if (MyCAN_RxMsg.IDE == CAN_Id_Standard)
    {
        OLED_ShowString(1, 6, "Std");

        OLED_ShowHexNum(2, 6, MyCAN_RxMsg.StdId, 8);
    }
    else if (MyCAN_RxMsg.IDE == CAN_Id_Extended)
    {
        OLED_ShowString(1, 6, "Ext");

        OLED_ShowHexNum(2, 6, MyCAN_RxMsg.ExtId, 8);
    }
    .....
}

```

10.数据传输策略

定时传输，发送方定时发，接收方直接接收（也可以过滤等等操作）

触发发送，发送方内部达到了条件就发送，适合一些警报和通知信息，我们之前演示的按键按下发送代码就是这类

请求发送，1.接受方接收请求数据，先发送一个遥控帧，如果发送方有这个数据帧，那么就发给他，当然也可以用数据帧请求数据帧，这更加有优势

2. **多请求问题**，多个节点同时请求一个数据，不能进行仲裁，尽量避免

1.发送,用定时器发送

定时传输比较占用资源，然后，发送越快就占用总线资源越多

对于id号比较小，优先级比较高的帧，不要让他们发送频率过高，否则低优先级可能很难抢到总线

```
CanTxMsg TxMsg_Timing = {
    .StdId = 0x100,
    .ExtId = 0x00000000,
    .IDE = CAN_Id_Standard,
    .RTR = CAN_RTR_Data,
    .DLC = 4,
    .Data = {0x11, 0x22, 0x33, 0x44}
};
/*定时发送*/
if (TimingFlag == 1)
{
    TimingFlag = 0;

    TxMsg_Timing.Data[0] ++;
    TxMsg_Timing.Data[1] ++;
    TxMsg_Timing.Data[2] ++;
    TxMsg_Timing.Data[3] ++;

    MyCAN_Transmit(&TxMsg_Timing);

    OLED_ShowHexNum(2, 5, TxMsg_Timing.Data[0], 2);
    OLED_ShowHexNum(2, 8, TxMsg_Timing.Data[1], 2);
    OLED_ShowHexNum(2, 11, TxMsg_Timing.Data[2], 2);
    OLED_ShowHexNum(2, 14, TxMsg_Timing.Data[3], 2);
    //这里没有用delay延时，避免阻塞，用定时器
}
```

定时器

```
void Timer_Init(void)
{
    RCC_APB1PeriphClockCmd(RCC_APB1Periph_TIM2, ENABLE);
```

```

TIM_InternalClockConfig(TIM2);

TIM_TimeBaseInitTypeDef TIM_TimeBaseInitStructure;
TIM_TimeBaseInitStructure.TIM_ClockDivision = TIM_CKD_DIV1;
TIM_TimeBaseInitStructure.TIM_CounterMode = TIM_CounterMode_Up;
TIM_TimeBaseInitStructure.TIM_Period = 1000 - 1;
TIM_TimeBaseInitStructure.TIM_Prescaler = 7200 - 1;
TIM_TimeBaseInitStructure.TIM_RepetitionCounter = 0;
TIM_TimeBaseInit(TIM2, &TIM_TimeBaseInitStructure);

TIM_ClearFlag(TIM2, TIM_FLAG_Update);
TIM_ITConfig(TIM2, TIM_IT_Update, ENABLE);

NVIC_PriorityGroupConfig(NVIC_PriorityGroup_2);

NVIC_InitTypeDef NVIC_InitStructure;
NVIC_InitStructure.NVIC_IRQChannel = TIM2_IRQn;
NVIC_InitStructure.NVIC_IRQChannelCmd = ENABLE;
NVIC_InitStructure.NVIC_IRQChannelPreemptionPriority = 2;
NVIC_InitStructure.NVIC_IRQChannelSubPriority = 1;
NVIC_Init(&NVIC_InitStructure);

TIM_Cmd(TIM2, ENABLE);
}

void TIM2_IRQHandler(void)
{
    //每100ms就自动执行定时操作
    if (TIM_GetITStatus(TIM2, TIM_IT_Update) == SET)
    {
        TimingFlag = 1;
        //不要在中断函数里调用CAN的发送函数还有oled的函数，因为会造成资源冲突，中断是快进快出
        TIM_ClearITPendingBit(TIM2, TIM_IT_Update);
    }
}

```

接收

```

/*接收部分*/
if (MyCAN_ReceiveFlag())
{
    MyCAN_Receive(&RxMsg);

    if (RxMsg.RTR == CAN_RTR_Data)
    {
        /*收到定时数据帧*/
        if (RxMsg.StdId == 0x100 && RxMsg.IDE == CAN_Id_Standard)
        //这个地方都用StdId == 0x100了，不就说明它已经是标准帧了吗，
        //为什么后面还要判断是不是标准帧
        //之前的代码讲过，stdid和extid可以都有值的，
        //真正决定标准格式还是扩展格式还是得靠IDE
        //扩展帧的std ID也可能是0x100
        {
            OLED_ShowHexNum(2, 5, RxMsg.Data[0], 2);
            OLED_ShowHexNum(2, 8, RxMsg.Data[1], 2);
        }
    }
}

```

```
        OLED_ShowHexNum(2, 11, RxMsg.Data[2], 2);
        OLED_ShowHexNum(2, 14, RxMsg.Data[3], 2);
    }
}
```

2.触发传输

发送方

```
CanTxMsg TxMsg_Trigger = { //触发发送结构体
    .StdId = 0x200,
    .ExtId = 0x00000000,
    .IDE = CAN_Id_Standard,
    .RTR = CAN_RTR_Data,
    .DLC = 4,
    .Data = {0x11, 0x22, 0x33, 0x44}
};

/*触发发送*/
KeyNum = Key_GetNum();
if (KeyNum == 1)
{
    TriggerFlag = 1;
}

if (TriggerFlag == 1)
{
    TriggerFlag = 0;

    TxMsg_Trigger.Data[0] ++;
    TxMsg_Trigger.Data[1] ++;
    TxMsg_Trigger.Data[2] ++;
    TxMsg_Trigger.Data[3] ++;

    MyCAN_Transmit(&TxMsg_Trigger);

    OLED_ShowHexNum(3, 5, TxMsg_Trigger.Data[0], 2);
    OLED_ShowHexNum(3, 8, TxMsg_Trigger.Data[1], 2);
    OLED_ShowHexNum(3, 11, TxMsg_Trigger.Data[2], 2);
    OLED_ShowHexNum(3, 14, TxMsg_Trigger.Data[3], 2);
}
}
```

接收方

```
/*收到触发数据帧*/
```

```
if (RxMsg.StdId == 0x200 && RxMsg.IDE == CAN_Id_Standard)
{
    OLED_ShowHexNum(3, 5, RxMsg.Data[0], 2);
    OLED_ShowHexNum(3, 8, RxMsg.Data[1], 2);
    OLED_ShowHexNum(3, 11, RxMsg.Data[2], 2);
    OLED_ShowHexNum(3, 14, RxMsg.Data[3], 2);
}
```

3.请求传输

发送方,可以收到遥控帧和数据帧请求, 数据帧的请求的数据可以作为请求参数

```
CanTxMsg TxMsg_Request = { //请求传输结构体
```

```
.StdId = 0x300,
.ExtId = 0x00000000,
.IDE = CAN_Id_Standard,
.RTR = CAN_RTR_Data,
.DLC = 4,
.Data = {0x11, 0x22, 0x33, 0x44}
```

```
};
```

```
CanRxMsg RxMsg;
```

```
/*请求发送*/
```

```
if (MyCAN_ReceiveFlag())
{
    MyCAN_Receive(&RxMsg);

    if (RxMsg.IDE == CAN_Id_Standard &&
        RxMsg.RTR == CAN_RTR_Remote &&
        RxMsg.StdId == 0x300) //请求收到标准id300的遥控帧, 回复一个他的数据帧,
    {
        RequestFlag = 1;
    }

    if (RxMsg.IDE == CAN_Id_Standard &&
        RxMsg.RTR == CAN_RTR_Data &&
        RxMsg.StdId == 0x3FF) //收到数据帧也可以的啊
    {
        //还可以把RxMsg.Data拿出来作为请求参数, 实现不同的子请求
        RequestFlag = 1;
    }
}

if (RequestFlag == 1)
{
    RequestFlag = 0;

    TxMsg_Request.Data[0] ++;
    TxMsg_Request.Data[1] ++;
```

```

        TxMsg_Request.Data[2] ++;
        TxMsg_Request.Data[3] ++;

        MyCAN_Transmit(&TxMsg_Request);

        OLED_ShowHexNum(4, 5, TxMsg_Request.Data[0], 2);
        OLED_ShowHexNum(4, 8, TxMsg_Request.Data[1], 2);
        OLED_ShowHexNum(4, 11, TxMsg_Request.Data[2], 2);
        OLED_ShowHexNum(4, 14, TxMsg_Request.Data[3], 2);
    }
}

```

接收方

```

CanTxMsg TxMsg_Request_Remote = {
    .StdId = 0x300,
    .ExtId = 0x00000000,
    .IDE = CAN_Id_Standard,
    .RTR = CAN_RTR_Remote,
    .DLC = 0,
    .Data = {0x00}
};

CanTxMsg TxMsg_Request_Data = {
    .StdId = 0x3FF,
    .ExtId = 0x00000000,
    .IDE = CAN_Id_Standard,
    .RTR = CAN_RTR_Data,
    .DLC = 0,
    .Data = {0x00}
};

CanRxMsg RxMsg;

main:

while (1)
{
    /*请求部分*/
    KeyNum = Key_GetNum();
    if (KeyNum == 1)
    {
        MyCAN_Transmit(&TxMsg_Request_Remote);
    }
    if (KeyNum == 2)
    {
        MyCAN_Transmit(&TxMsg_Request_Data);
    }

    /*接收部分*/
    if (MyCAN_ReceiveFlag())
    {
        MyCAN_Receive(&RxMsg);
    }
}

```



```
/*收到请求数据帧*/  
    if (RxMsg.StdId == 0x300 && RxMsg.IDE == CAN_Id_Standard)  
    {  
        OLED_ShowHexNum(4, 5, RxMsg.Data[0], 2);  
        OLED_ShowHexNum(4, 8, RxMsg.Data[1], 2);  
        OLED_ShowHexNum(4, 11, RxMsg.Data[2], 2);  
        OLED_ShowHexNum(4, 14, RxMsg.Data[3], 2);  
    }  
}  
  
...
```

感恩的心，谢谢江科大，CAN入门教程，学明白了，那么下一步，CANopen...加油！
